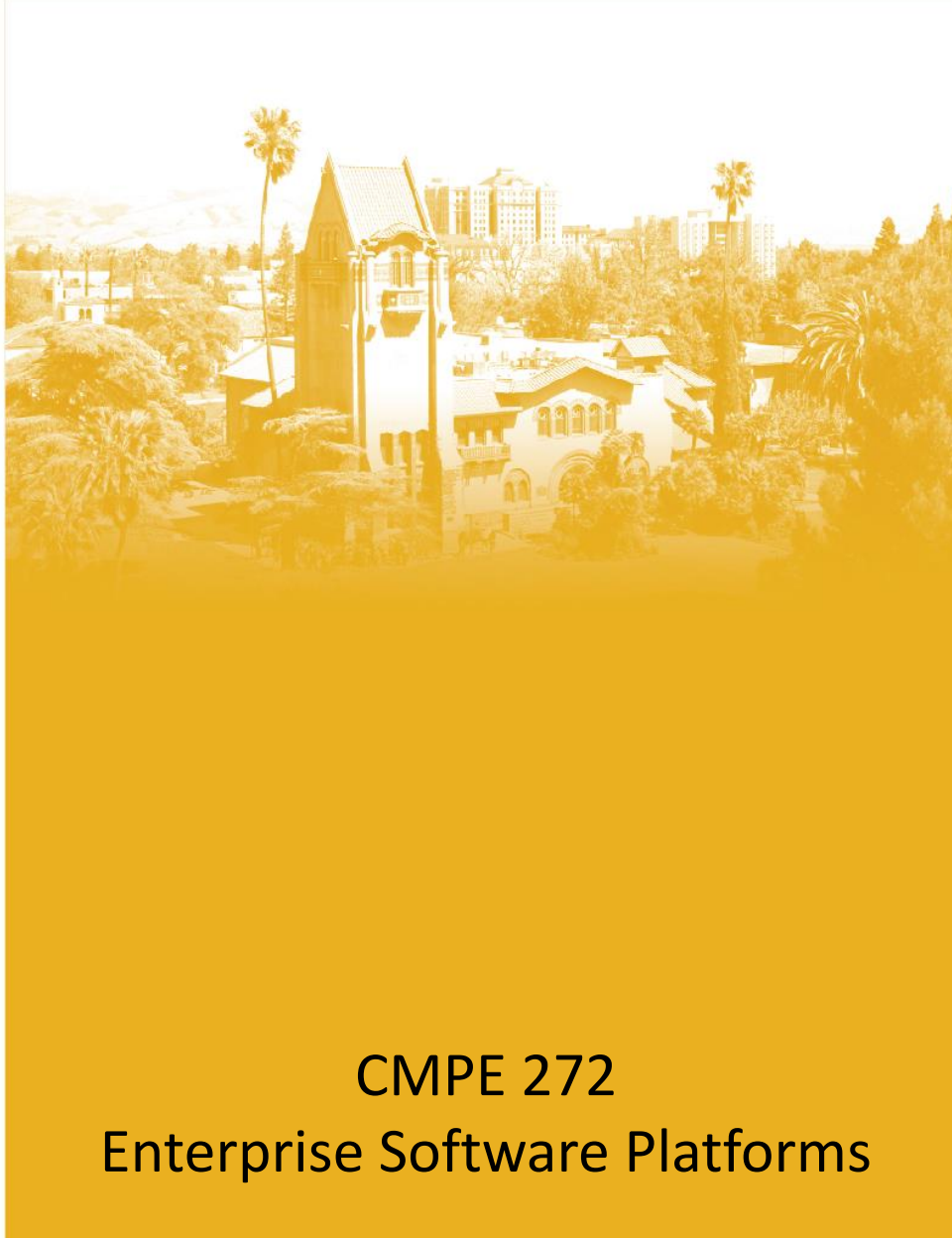




CMPE 272
Enterprise Software Platforms

Enterprise Software Platforms

Instructor: Andrew Bond

IT News — APIs and integration in 2026

Arazzo 1.1.0 (OpenAPI Initiative, May 2026) adds AsyncAPI support to API workflow descriptions.

A workflow step can publish an event, wait for a message, and correlate the response. One description spanning REST and events.

Apigee's MCP Gateway reached general availability in January 2026.

Existing enterprise APIs are exposed as Model Context Protocol tools for AI agents, with the gateway handling auth and governance.

API management and event streaming are converging.

Kong's Event Gateway and Gravitee's Kafka Gateway both put Kafka topics in the same developer portal as REST APIs, discovered and subscribed the same way.

The Kafka conversation has shifted from scaling to governance.

Tonight runs from CORBA to OpenAPI contracts. These three items are where that arc is heading: one platform for synchronous and asynchronous APIs.

Sources: openapis.org · redocly.com · cloud.google.com/apigee · konghq.com · gravitee.io

UNIT 2 · APPLICATION FRAMEWORKS AND APIS (WEEK 3)

- Legacy Client/Server (RPC, CORBA, SOAP, REST, Web Services, etc..) – SOA Framework and Architectures
- Modern Application Frameworks
 - Software Frameworks and Middleware
 - Specific Application Frameworks & Technologies
 - [Karaf](#) (Apache application container)
 - Camel (Apache implementation of the [Enterprise Integration Patterns](#))
 - [Ruby on Rails](#)
 - [Spring](#)
 - JavaScript Frameworks
 - Hadoop Frameworks
 - Microservice / Service Mesh



Inter-process communication Technology & Standards

Legacy

- [IPC](#)/RPC
- [CORBA](#)
- [Java RMI](#)
- [XML RPC](#) / [JSON RPC](#)
- [Microsoft .NET](#)
- [ODBC](#)

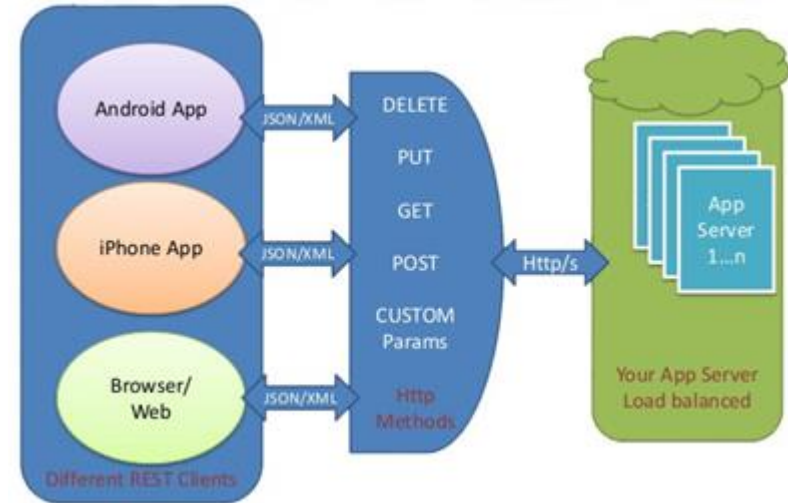
Modern

- World Wide Web Consortium (W3C)
- SOAP
- WSDL
- UDDI
- Apache Thrift
- REST ([Roy Fielding](#))
- WCF
[https://en.wikipedia.org/wiki/Windows Communication Foundation](https://en.wikipedia.org/wiki/Windows_Communication_Foundation)
- Google Protocol Buffers
- <http://pages.cs.wisc.edu/~remzi/OSTEP/dist-intro.pdf>

Representational state transfer (REST)

The architectural properties affected by the constraints of the REST architectural style are:

- **Performance** - component interactions can be the dominant factor in user-perceived performance and network efficiency.[6]
- **Scalability** to support large numbers of components and interactions among components
- **Simplicity** of interfaces
- **Modifiability** of components to meet changing needs (even while the application is running)
- **Visibility** of communication between components by service agents
- **Portability** of components by moving program code with the data
- **Reliability** is the resistance to failure at the system level in the presence of failures within components, connectors, or data



HTTP Verb	CRUD	Entire Collection (e.g. /customers)
POST	Create	201 (Created), 'Location' header with link to /customers/{id} containing new ID.
GET	Read	200 (OK), list of customers. Use pagination, sorting and filtering to navigate big lists.
PUT	Update/Replace	405 (Method Not Allowed), unless you want to update/replace every resource in the entire collection.
PATCH	Update/Modify	405 (Method Not Allowed), unless you want to modify the collection itself.
DELETE	Delete	405 (Method Not Allowed), unless you want to delete the whole collection—not often desirable.

RAML

- **RESTful API Modeling Language (RAML)** is a [YAML](#)-based language for describing [RESTful APIs](#)
- RAML provides all the information necessary to describe RESTful or practically-RESTful APIs
- API's structure is manifest and easily understood by everyone: developers, partners, and other API-consumers
- Example RAML (Jukebox API):
<https://raw.githubusercontent.com/raml-org/raml-tutorial-200/step8/jukebox-api.raml>

Similar to WSDL in SOAP

```
/albums:
  type:
    collection:
      exampleCollection: !include jukebox-include-albums.sample
      exampleItem: !include jukebox-include-album-new.sample
    get:
      is: [
        searchable: {description: "with valid searchable fields: genreCode",
          example: "[\"genreCode\", \"ELE\", \"equals\"]",
          orderable: {fieldsList: "albumName, genre"},
          pageable
        }
      ]
    /{albumId}:
      type:
        collection-item:
          exampleItem: !include jukebox-include-album-retrieve.sample
        /songs:
          type:
            readOnlyCollection:
              exampleCollection: !include jukebox-include-album-songs.sample
            get:
              is: [orderable: {fieldsList: "songTitle"}]
              description: Get the list of songs for the album with `albumId = {albumId}
```

REST Architectural Constraints

- **Client-server** – A uniform interface separates clients from servers
- **Stateless** – no client context being stored on the server between requests
- **Cacheable & Idempotent** – clients and intermediaries can cache responses
- **Layered system**
- **Code on demand (optional)**
- **Uniform interface**

Web service APIs that adhere to the REST architectural constraints are called **RESTful APIs**

Protocol Buffers

- Support for Java, C++, or Python
- See [tutorials](#) or delve deeper into [protocol buffer encoding](#). API [reference documentation](#) is also provided for all three languages, as well as [language](#) and [style](#) guides for writing .proto files
- Protocol buffers are a flexible, efficient, automated mechanism for serializing structured data – think XML, but smaller, faster, and simpler
- specify how you want the information you're serializing to be structured by defining protocol buffer message types in .proto files. Each protocol buffer message is a small logical record of information, containing a series of name-value pairs. Here's a very basic example of a .proto file that defines a message containing information about a person
- A compiler takes .proto files to generate data access classes. These provide simple accessors for each field (like name() and set_name()) as well as methods to serialize/parse the whole structure to/from raw bytes
- So, for instance, if your chosen language is C++, running the compiler on the above example will generate a class called Person. You can then use this class in your application to populate, serialize, and retrieve Person protocol buffer messages.

Protocol buffers:

- are simpler
- are 3 to 10 times smaller
- are 20 to 100 times faster
- are less ambiguous
- generate data access classes that are easier to use programmatically

<https://developers.google.com/protocol-buffers/docs/overview>

```
message Person {
    required string name = 1;
    required int32 id = 2;
    optional string email = 3;

    enum PhoneType {
        MOBILE = 0;
        HOME = 1;
        WORK = 2;
    }

    message PhoneNumber {
        required string number = 1;
        optional PhoneType type = 2 [default = HOME];
    }

    repeated PhoneNumber phone = 4;
}
```

```
Person person;
person.set_name("John Doe");
person.set_id(1234);
person.set_email("jdoe@example.com");
fstream output("myfile", ios::out | ios::binary);
person.SerializeToOstream(&output);
```

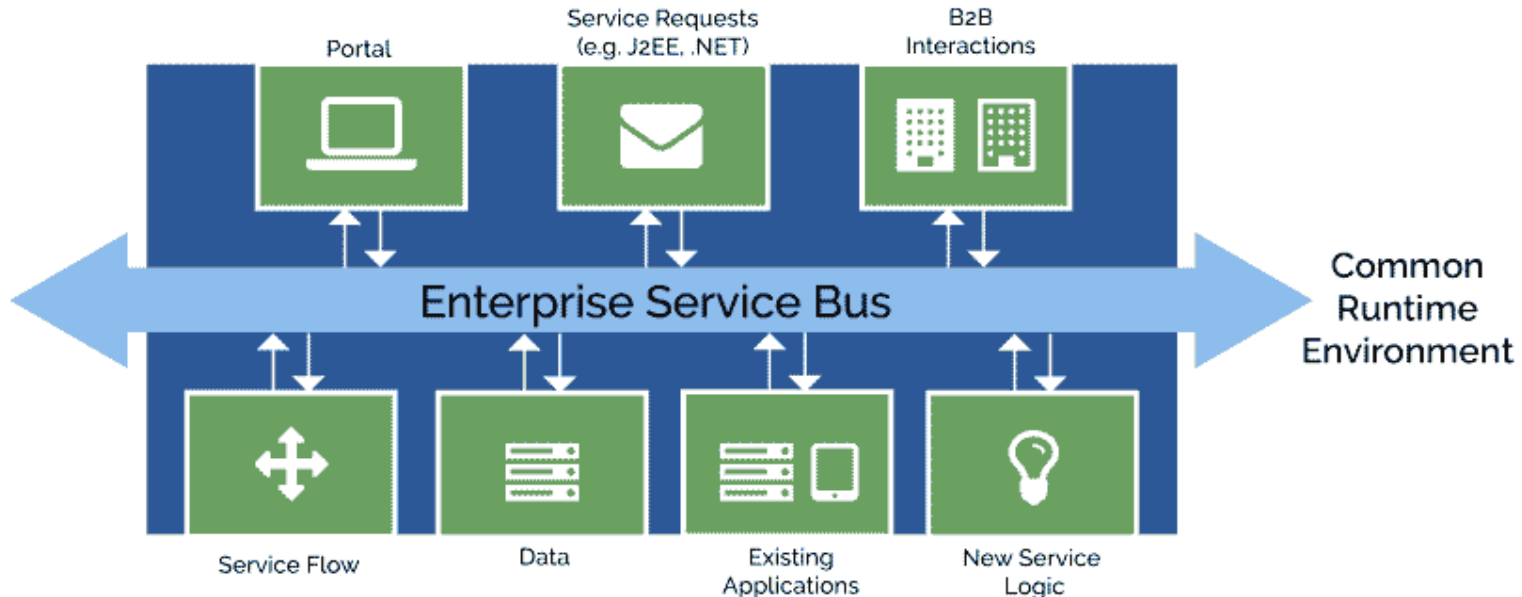
```
fstream input("myfile", ios::in | ios::binary);
Person person;
person.ParseFromIstream(&input);
cout << "Name: " << person.name() << endl;
cout << "E-mail: " << person.email() << endl;
```

gRPC (google Remote Procedure Call)

- **Protocol Buffers (Protobuf) for Serialization:**
 - gRPC uses **Protocol Buffers** (or Protobuf) as its Interface Definition Language (IDL) and for data serialization.
 - Using Protobuf allows gRPC to achieve smaller message sizes and faster transmission compared to traditional formats like JSON or XML.
- **Client-Server Architecture:**
 - **unary calls** (one request, one response)
 - **server streaming** (one request, multiple responses)
 - **client streaming** (multiple requests, one response),
 - **bidirectional streaming** (multiple requests, multiple responses).
- **HTTP/2 for Transport**
 - **Multiplexing:** Multiple requests and responses can be sent over the same connection simultaneously, without the overhead of opening new connections.
 - **Bidirectional streaming:** Allows real-time communication between client and server.
 - **Header compression:** Reduces the size of messages and improves performance, especially for services with many small requests and responses.
 - **Experimental support for gRPC over HTTP/3** is already available in some frameworks, but full production-ready implementations are still in progress

Enterprise service bus (ESB)

- A middleware tool used to distribute work among connected components of an application. ESBs are designed to provide a uniform means of moving work, offering applications the ability to connect to the bus and subscribe to messages based on simple structural and business policy rules



What do ESBs do?

Functionality:

- **Distribution of Work:** ESBs allow applications to offload tasks and distribute work among connected components, ensuring that the right component processes the right task.
- **Uniform Communication:** ESBs offer a consistent and standardized way of transferring messages between applications, irrespective of their underlying technology or platform.

Connectivity:

- **Subscription Model:** Applications can connect to the ESB and subscribe to specific messages. This is based on structural criteria or business rules, ensuring that only relevant messages are delivered to the appropriate application.
- **Loose Coupling:** ESBs enable applications to interact without having a direct connection, promoting flexibility and scalability. Changes in one application don't necessarily impact others connected to the ESB.

Advantages:

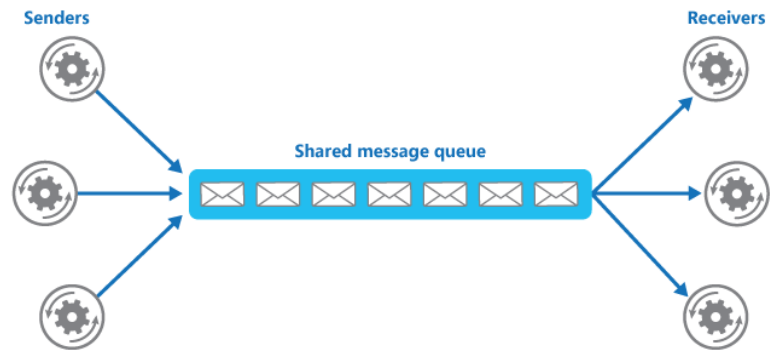
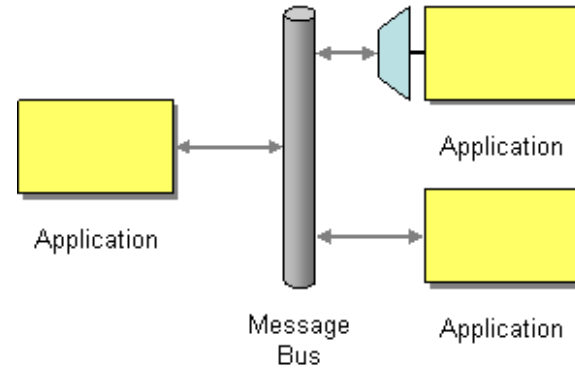
- **Interoperability:** ESBs allow different applications, often built on varied technologies, to communicate seamlessly.
- **Scalability:** As the business grows, new components can be easily added to the system without much reconfiguration.
- **Flexibility:** ESBs support dynamic routing and transformation of messages, adapting to changing business needs.

Use Cases:

- **Business Process Automation:** Streamlining business processes by coordinating tasks between different applications.
- **System Integration:** Connecting legacy systems with modern applications, ensuring seamless data flow.
- **Real-time Analytics:** Gathering data from different sources in real-time for analysis.

Message Bus / Message Queue

- A **Message Bus** is a messaging infrastructure to allow different systems to communicate through a **shared set of interfaces (message bus)**.
- **Message queue** Two (or more) processes can exchange information via **access to a common system message queue**.
- The sending process places via some (OS) message-passing module a message onto a queue which can be read by another process



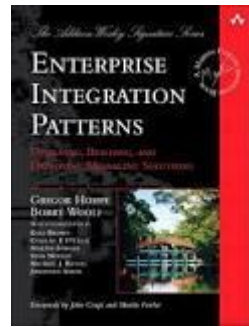
Integration Patterns

- Camel supports most of the [Enterprise Integration Patterns](#) from the excellent book by [Gregor Hohpe](#) and [Bobby Woolf](#).
- If you are new to Camel you might want to try the [Getting Started](#) in the [User Guide](#) before attempting to implement these patterns.
- <http://camel.apache.org/enterprise-integration-patterns.html>
- [Enterprise Integration Patterns Using Mule](#)
- [Camunda Workflow Patterns](#)

Integration Patterns in Enterprise Software Platforms

- **Foundational Knowledge:** Integration Patterns provide the foundational knowledge required for students to understand how different software components communicate, interact, and work cohesively within an enterprise ecosystem.
- **Standardization:** They offer standardized solutions to common integration challenges. By teaching these patterns, students learn tried-and-true methods rather than reinventing the wheel.
- **Complex System Navigation:** Enterprise systems are often intricate and multifaceted. Integration Patterns equip students with tools to deconstruct complexity, making it easier to design, implement, and manage integrations.
- **Scalability & Evolution:** As enterprise platforms evolve and scale, integration needs change. Understanding these patterns ensures that students can design systems that are both robust and adaptable to changing requirements.
- **Real-world Relevance:** Given that many businesses utilize enterprise software platforms, having knowledge of Integration Patterns prepares students for real-world challenges, making them more marketable and relevant in the job market.
- **Enhanced Problem Solving:** By studying various patterns, students develop a broader perspective and a toolkit to approach integration problems, fostering enhanced problem-solving skills.
- **Interoperability:** In today's diverse technological landscape, ensuring different systems work together is crucial. Integration Patterns teach students the best practices to achieve seamless interoperability.

Enterprise Integration Patterns (EIP) - The Book



Integration Platforms:

- IBM WebSphere MQ
- TIBCO
- Vitria
- WebMethods (Software AG)
- Microsoft BizTalk

Messaging systems

- JMS
- WCF
- Rabbit MQ or MSMQ

ESBs

- Apache Camel
- Mule
- WSO2
- Oracle Service Bus
- Open ESB
- SonicMQ
- Fiorano or Fuse
- ServiceMix

Enterprise Integration Patterns

- **Pattern Language:** The authors introduce a "pattern language" for integration, which is a set of patterns that describe how to design, build, and deploy integration solutions.
- **Messaging Systems:** The book delves into the advantages of using asynchronous messaging architectures in an enterprise setting, emphasizing reliability, scalability, and maintainability.
- **Core Patterns:** The authors discuss over 60 patterns related to messaging, including:
 - **Message Channel:** How applications connect to each other through a channel.
 - **Message Router:** Directing a message to a specific location based on a set of conditions.
 - **Message Translator:** Converting message formats between systems.
 - **Message Endpoint:** How an application connected to a messaging channel interacts with it to send or receive messages.

UI FRAMEWORKS & TECHNOLOGY

Javascript Libraries



[jQuery](#) is the most popular JavaScript framework on the Internet today.

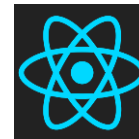
- It uses CSS selectors to access and manipulate HTML elements (DOM Objects) on a web page.
- jQuery also provides a companion UI (user interface) framework and numerous other plug-ins.
- **Prototype** is a JavaScript library that provides a simple API to perform common web tasks.
- **API** is short for Application Programming Interface. It is a library of properties and methods for manipulating the HTML DOM.
- Prototype enhances JavaScript by providing classes and inheritance.



[MooTools](#) is also a framework that offers an API to make common JavaScript programming easier.

- MooTools also includes some lightweight effects and animation functions.

[React.js](#) is a JavaScript library for building user interfaces



- Provides a view for Data rendered as HTML
- Maintained by Facebook, Instagram and community

Other Frameworks

- **YUI** - The Yahoo! User Interface Framework is a large library that covers a lot of functions, from simple JavaScript utilities to complete internet widgets. [[DISCONTINUED](#)]
- [Ext JS](#) - Customizable widgets for building rich Internet applications.
- [Dojo](#) - A toolkit designed around packages for DOM manipulation, events, widgets, and more.
- [script.aculo.us](#) - Open-source JavaScript framework for visual effects and interface behaviors
- [Midori](#) - Ultra-lightweight JavaScript framework



Popular Javascript Frameworks



- [Bootstrap](#) – “Sleek, intuitive, and powerful front-end framework for faster and easier web development.” (Internal Twitter framework)
- [Foundation](#) – a family of responsive front-end frameworks that make it easy to design beautiful responsive websites, apps and emails that look amazing on any device



- [AngularJS](#) – Superheroic JavaScript MVW Framework – [tutorial](#)
- [MEAN stack](#), consisting of [MongoDB](#) database, [Express.js](#) web application server framework, Angular.js itself, and [Node.js](#) server runtime environment



- [React.js](#) – Declarative, Component-Based, Addresses challenges encountered in developing [single-page applications](#)



- [Vue.js](#) - best from Ember, React and Angular, putting all that into a handy package. It is proved to be faster and leaner, comparing to React and Angular 2.0



- [Node.js](#) - built on [Chrome's V8 JavaScript engine](#). Node.js uses an event-driven, non-blocking I/O model that makes it lightweight and efficient. Node.js' package ecosystem, [npm](#), is the largest ecosystem of open source libraries in the world

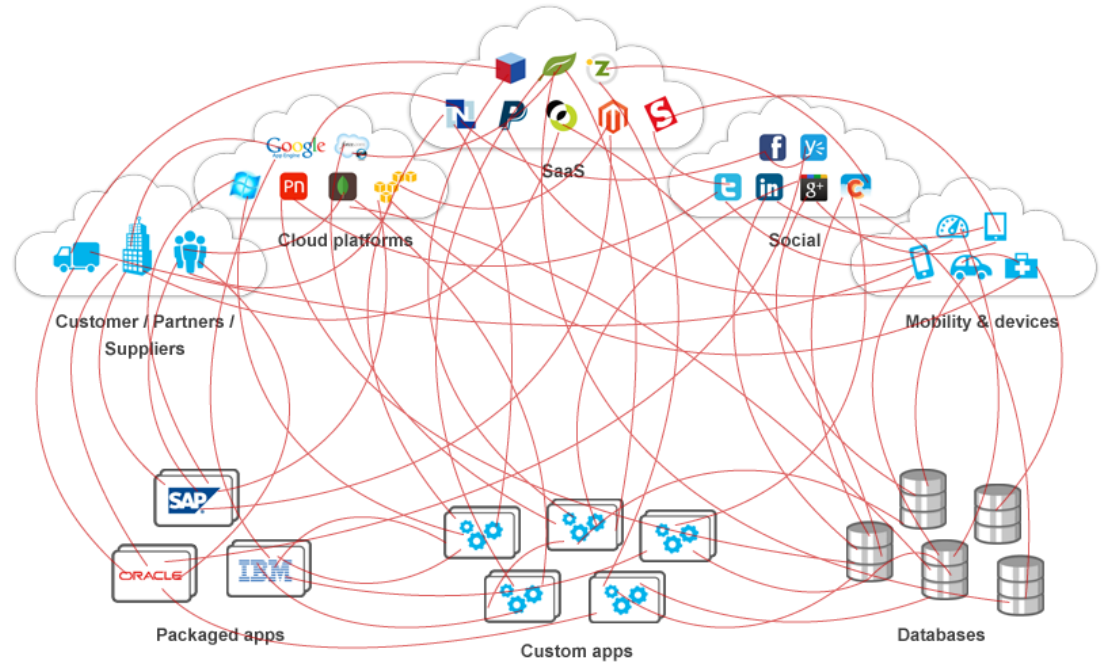


- [Ember.js](#) – A framework for ambitious web applications

Javascript Programming

- <http://www.w3schools.com/js/>
- <https://www.codecademy.com/en/tracks/javascript>
- <https://www.coursera.org/courses?query=javascript>

J2EE
OSGi
Apache Camel
Spring
Mule
Microsoft
Frameworks
Web Frameworks
(AWS, etc..)



TRADITIONAL APPLICATION PLATFORMS & FRAMEWORKS

Java/Jakarta Platform, Enterprise Edition

- Java Platform, Enterprise Edition or Java EE is a widely used enterprise computing platform developed under the Java Community Process
- The platform provides an API and runtime environment for developing and running enterprise software, including network and web services, and other large-scale, multi-tiered, scalable, reliable, and secure network applications
- Java EE extends the Java Platform, Standard Edition (Java SE), providing an API for object-relational mapping, distributed and multi-tier architectures, and web services
- The platform incorporates a design based largely on modular components running on an application server. Software for Java EE is primarily developed in the Java programming language
- The platform emphasizes convention over configuration and annotations for configuration
- Optionally XML can be used to override annotations or to deviate from the platform defaults.
- Versions
 - J2EE 1.2 (December 12, 1999)
 - J2EE 1.3 (September 24, 2001)
 - J2EE 1.4 (November 11, 2003)
 - Java EE 5 (May 11, 2006)
 - Java EE 6 (December 10, 2009)
 - Java EE 7 (May 28, 2013)
 - Java EE 8 (August 31, 2017)
 - Jakarta EE 8 (September 10, 2019)
 - Jakarta EE 9 (November 22 2020)
 - Jakarta EE 9.1 (May 25 2021)



J2EE Application Servers

- [Apache Tomcat](#)
- [GlassFish](#) server Open Source Edition
- [Oracle WebLogic Server](#)
- [IBM WebSphere Application Server](#)
- [JBoss Enterprise Application Platform](#)
- [Eclipse Jetty](#): A smaller and more specialized server that focuses on HTTP services and can handle a wide array of Java EE specifications like Servlets and WebSockets.
- [WildFly](#): A successor to JBoss AS (Application Server), WildFly is open-source and supports the latest Jakarta EE specifications.



ORACLE



WebSphere® software

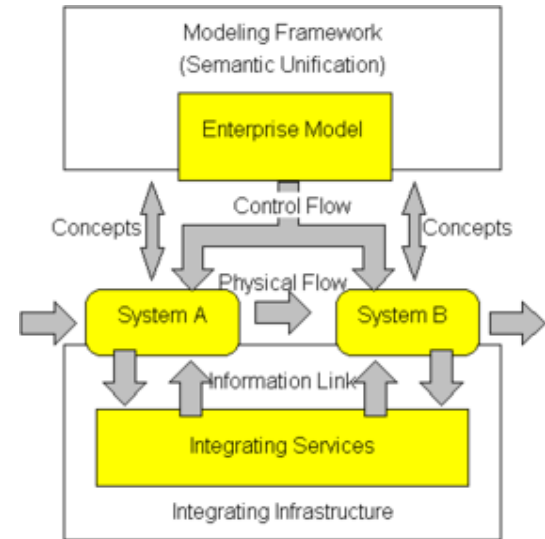


jetty://



Enterprise Integration

- Enabling integration of systems and applications across an enterprise
 - System interconnection
 - Electronic data interchange
 - Product data exchange
 - Distributed computing environments



Wiring Processors and Endpoints

•Java DSL

```
from ("file:/tmp").to("jms:aQueue");
```

•Spring DSL

```
<route>  
  <from uri="file:/tmp"/>  
    <to uri="jms:aQueue"/>  
</route>
```

•Scala DSL

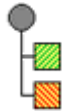
```
from "file:/tmp" -> "jms:aQueue"
```

Messaging Systems

Camel supports most of the [Enterprise Integration Patterns](#)



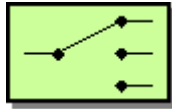
[Message Channel](#)



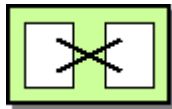
[Message](#)



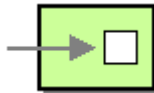
[Pipes and Filters](#)



[Message Router](#)



[Message Translator](#)



[Message Endpoint](#)

How does one application communicate with another using messaging?

How can two applications connected by a message channel exchange a piece of information?

How can we perform complex processing on a message while maintaining independence and flexibility?

How can you decouple individual processing steps so that messages can be passed to different filters depending on a set of conditions?

How can systems using different data formats communicate with each other using messaging?

How does an application connect to a messaging channel to send and receive messages?



Messaging Channels



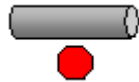
[Point to Point Channel](#)

How can the caller be sure that exactly one receiver will receive the document or perform the call?



[Publish Subscribe Channel](#)

How can the sender broadcast an event to all interested receivers?



[Dead Letter Channel](#)

What will the messaging system do with a message it cannot deliver?



[Guaranteed Delivery](#)

How can the sender make sure that a message will be delivered, even if the messaging system fails?



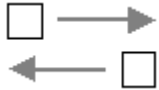
[Message Bus](#)

What is an architecture that enables separate applications to work together, but in a de-coupled fashion such that applications can be easily added or removed without affecting the others?

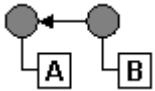
Message Construction



[Event Message](#)



[Request Reply](#)



[Correlation Identifier](#)



[Return Address](#)

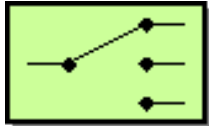
How can messaging be used to transmit events from one application to another?

When an application sends a message, how can it get a response from the receiver?

How does a requestor that has received a reply know which request this is the reply for?

How does a replier know where to send the reply?

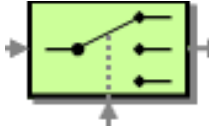
Message Routing



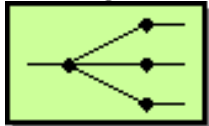
[Content Based Router](#)



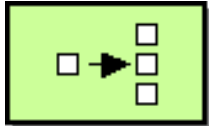
[Message Filter](#)



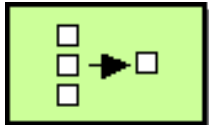
[Dynamic Router](#)



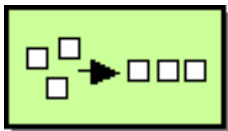
[Recipient List](#)



[Splitter](#)



[Aggregator](#)



[Resequencer](#)

How do we handle a situation where the implementation of a single logical function (e.g., inventory check) is spread across multiple physical systems?

How can a component avoid receiving uninteresting messages?

How can you avoid the dependency of the router on all possible destinations while maintaining its efficiency?

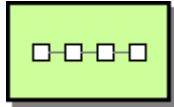
How do we route a message to a list of (static or dynamically) specified recipients?

How can we process a message if it contains multiple elements, each of which may have to be processed in a different way?

How do we combine the results of individual, but related messages so that they can be processed as a whole?

How can we get a stream of related but out-of-sequence messages back into the correct order?

Message Routing, Continued

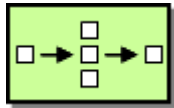


[Composed Message Processor](#)

How can you maintain the overall message flow when processing a message consisting of multiple elements, each of which may require different processing?

[Scatter-Gather](#)

How do you maintain the overall message flow when a message needs to be sent to multiple recipients, each of which may send a reply?



[Routing Slip](#)

How do we route a message consecutively through a series of processing steps when the sequence of steps is not known at design-time and may vary for each message?

[Throttler](#)

How can I throttle messages to ensure that a specific endpoint does not get overloaded, or we don't exceed an agreed SLA with some external service?

[Sampling](#)

How can I sample one message out of many in a given period to avoid downstream route does not get overloaded?

[Delayer](#)

How can I delay the sending of a message?

[Load Balancer](#)

How can I balance load across a number of endpoints?

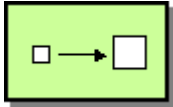
[Multicast](#)

How can I route a message to a number of endpoints at the same time?

[Loop](#)

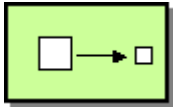
How can I repeat processing a message in a loop?

Message Transformation



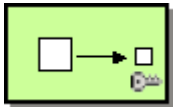
[Content Enricher](#)

How do we communicate with another system if the message originator does not have all the required data items available?



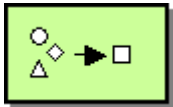
[Content Filter](#)

How do you simplify dealing with a large message, when you are interested only in a few data items?



[Claim Check](#)

How can we reduce the data volume of message sent across the system without sacrificing information content?



[Normalizer](#)

How do you process messages that are semantically equivalent, but arrive in a different format?

[Sort](#)

How can I sort the body of a message?

[Script](#)

How do I execute a script which may not change the message?

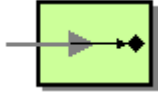
[Validate](#)

How can I validate a message?

Messaging Endpoints

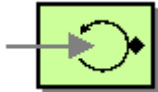
[Messaging Mapper](#)

How do you move data between domain objects and the messaging infrastructure while keeping the two independent of each other?



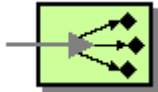
[Event Driven Consumer](#)

How can an application automatically consume messages as they become available?



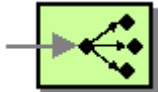
[Polling Consumer](#)

How can an application consume a message when the application is ready?



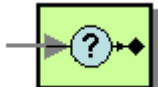
[Competing Consumers](#)

How can a messaging client process multiple messages concurrently?



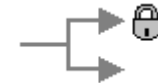
[Message Dispatcher](#)

How can multiple consumers on a single channel coordinate their message processing?



[Selective Consumer](#)

How can a message consumer select which messages it wishes to receive?



[Durable Subscriber](#)

How can a subscriber avoid missing messages while it's not listening for them?



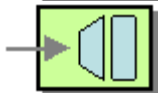
[Idempotent Consumer](#)

How can a message receiver deal with duplicate messages?



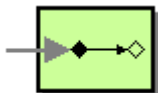
[Transactional Client](#)

How can a client control its transactions with the messaging system?



[Messaging Gateway](#)

How do you encapsulate access to the messaging system from the rest of the application?



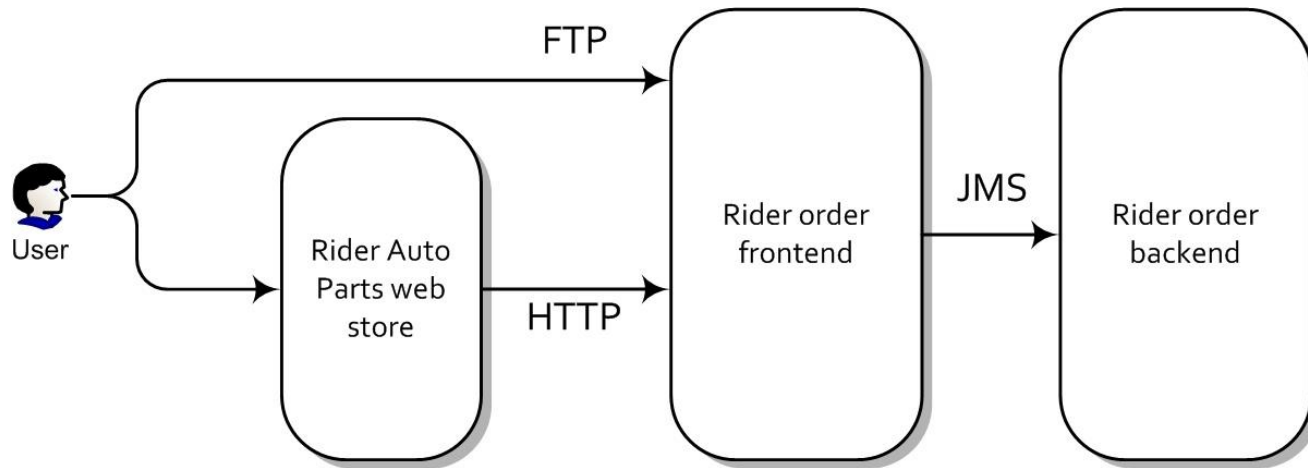
[Service Activator](#)

How can an application design a service to be invoked both via various messaging technologies and via non-messaging

Example: “Rider Auto Parts”

- Over the years, they’ve changed the way they receive orders several times
- Initially, orders were placed by uploading comma-separated value (CSV) files to an FTP server
- The message format was later changed to XML
- Currently they provide a website through which orders are submitted as XML messages over HTTP
- New customers use the web interface to place orders, but because of service level agreements (SLAs) with existing customers, they must keep all the old message formats and interfaces up and running
- All of these messages are converted to an internal Plain Old Java Object (POJO) format before processing

High level view of the order processing system



Patterns in-use

1. There are two Message Endpoints; one for FTP connectivity and another for HTTP.

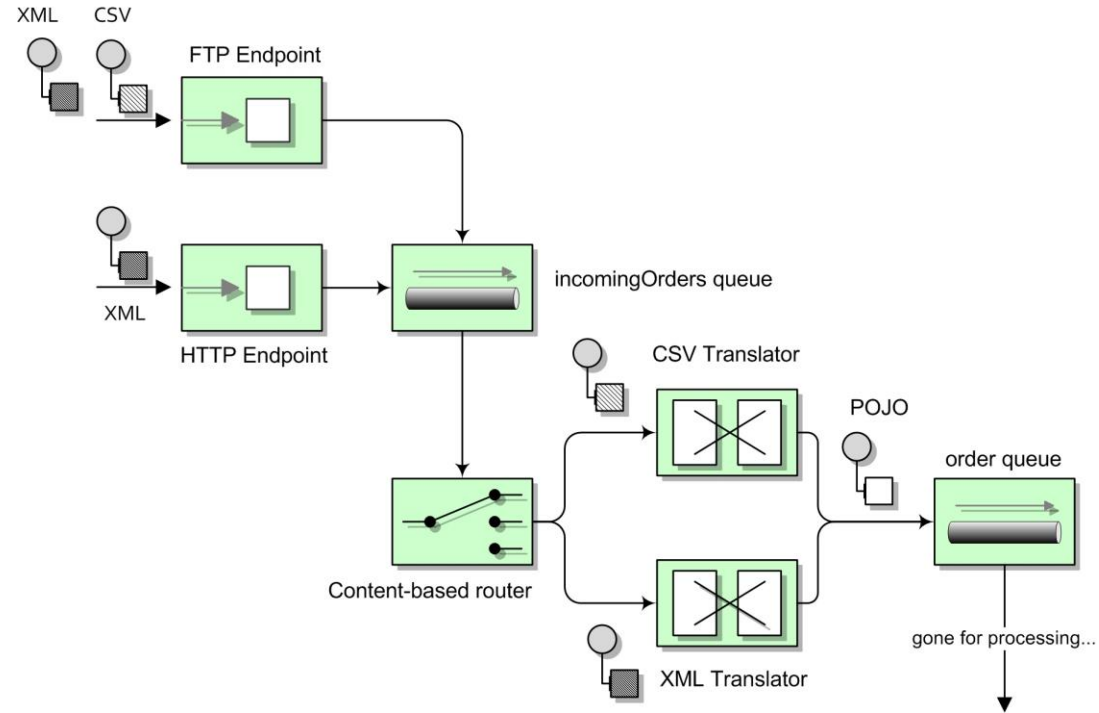
2. Messages from these endpoints are fed into the *incomingOrders* Message Channel

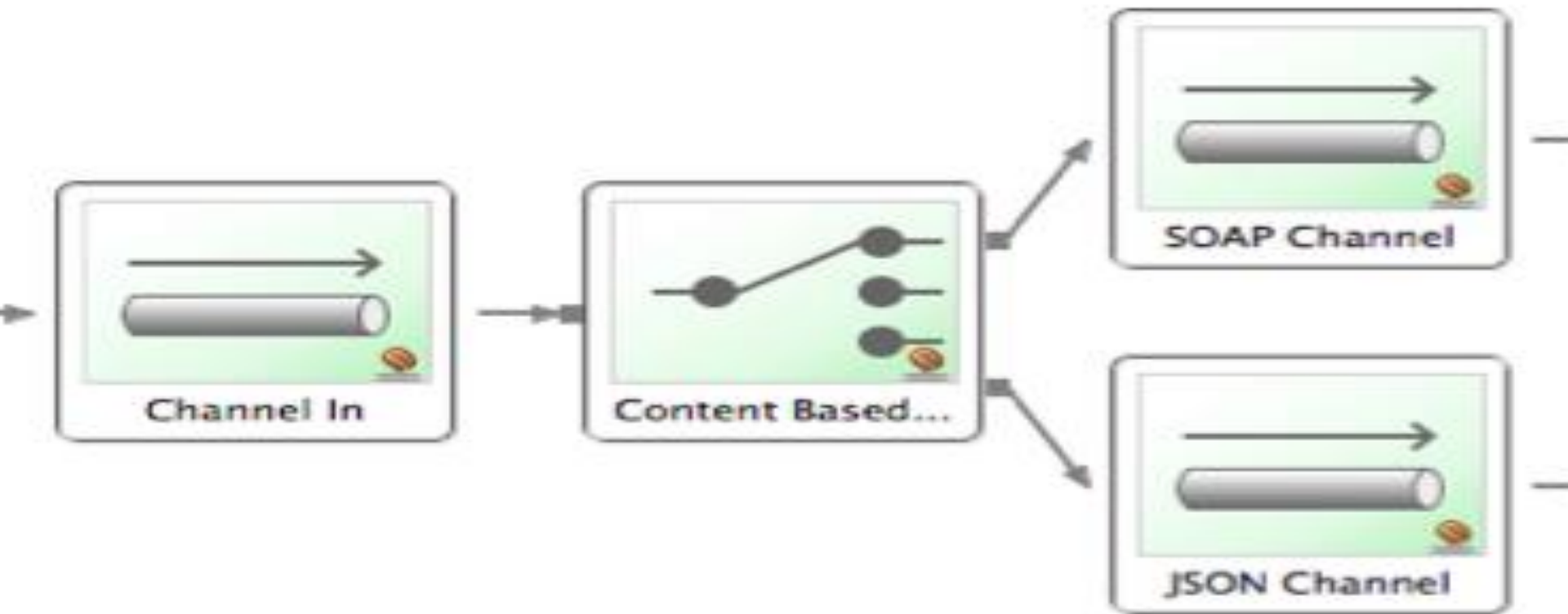
3. The messages are consumed from the *incomingOrders* Message Channel and routed by a Content-Based Router to one of two Message Translators. As the EIP name implies, the routing destination depends on the content of the message. In this case we need to route based on whether the content is a CSV or XML file.

4. Both Message Translators convert the message content into a POJO, which is fed into the *orders* Message Channel.

The whole section that uses a Content-Based Router and several Message Translators is referred to as a Normalizer.

Solution using EIPs





OTHER INTEGRATION FRAMEWORKS

Spring Framework

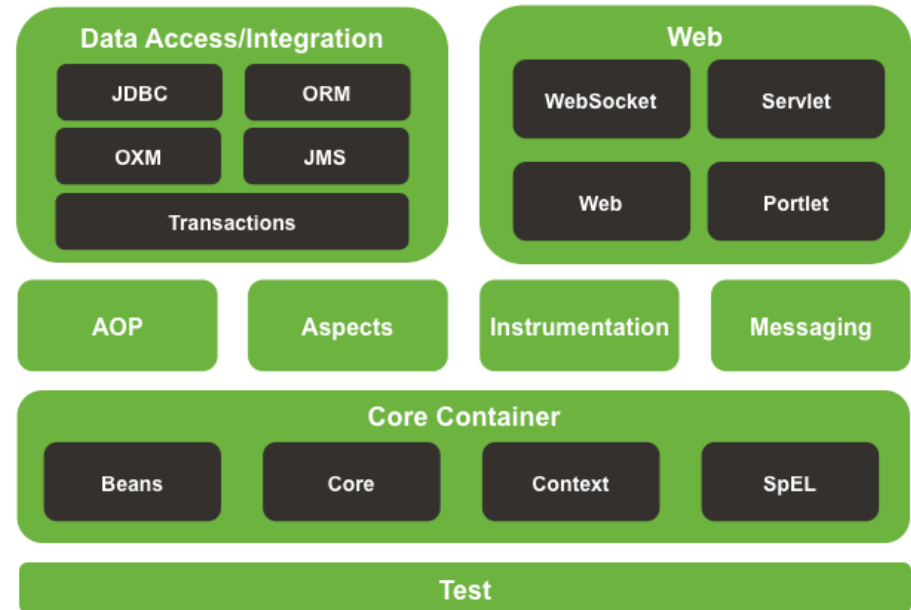


The [Spring Framework](#) is an application **framework** and inversion of control container for the Java platform.

The **framework's** core features provide a comprehensive programming and configuration model for modern Java-based enterprise applications



Spring Framework Runtime



What is the Spring Framework?

<http://spring.io/docs>

Commercial App Frameworks

- [Oracle Application Development Framework - Oracle ADF](#)
- Salesforce ([Lightning](#), [Heroku](#), [Informatica Cloud](#))
- IBM
- [Mulesoft](#)
- Dell (Boomi)

[Salesforce Lightning Demo](#)

Inversion Of Control

Let's consider a simple example. Imagine we're writing a program to get some information from a user via a command line enquiry, like this:

```
#ruby
puts 'What is your name?'
name = gets
process_name(name)
puts 'What is your quest?'
quest = gets
process_quest(quest)
```

In a windowing system to do something like this, we would do it by configuring a window:

```
require 'tk'
root = TkRoot.new()
name_label = TkLabel.new() {text "What is
Your Name?"}
name_label.pack
name = TkEntry.new(root).pack
name.bind("FocusOut") {process_name(name)}
quest_label = TkLabel.new() {text "What is
Your Quest?"}
quest_label.pack
quest = TkEntry.new(root).pack
quest.bind("FocusOut")
{process_quest(quest)}
Tk.mainloop()
```

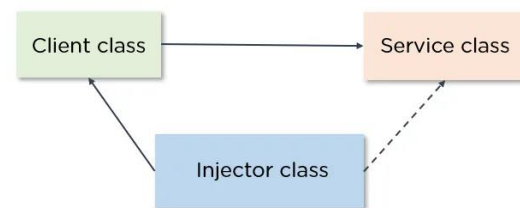
- Difference in the flow of control between these programs - in particular the control of when the `process_name` and `process_quest` methods are called
- In the command line form I control when these methods are called, but in the window example I don't -> Instead I hand control over to the windowing system
- The Windowing System then decides when to call my methods, based on the bindings I made when creating the form

One important characteristic of a framework is that the methods defined by the user to tailor the framework will often be called from within the framework itself, rather than from the user's application code. The framework often plays the role of the main program in coordinating and sequencing application activity. This inversion of control gives frameworks the power to serve as extensible skeletons. The methods supplied by the user tailor the generic algorithms defined in the framework for a particular application.

-- Ralph Johnson and Brian Foote

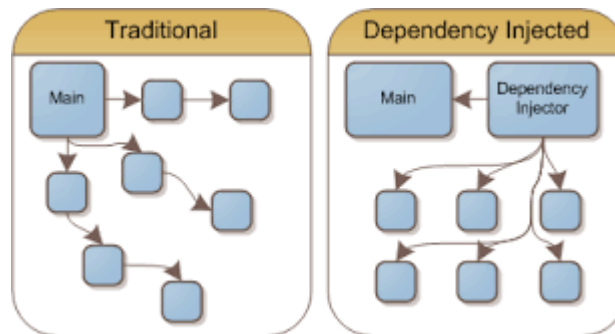
Dependency Injection (DI)

What It Is: A technique where an object's dependencies are provided externally rather than the object creating them internally. This is often considered a specific implementation of IoC.



Types:

- **Constructor Injection:** Dependencies are passed via the constructor.
- **Setter Injection:** Dependencies are provided through setter methods.
- **Interface Injection:** Dependencies are injected through methods defined in an interface.



<https://martinfowler.com/articles/injection.html>

<https://medium.com/@hyleedevlop/ios-swift-dependency-injection-di-and-dependency-inversion-principle-dip-64dd543238ca>

Event-Driven Architecture (EDA)

Events:

- **Definition:** An event represents a significant change in the state of a system. It could be a user action (e.g., clicking a button), a sensor reading, or a system update (e.g., a completed transaction).
- **Event Types:** Events can be classified into two broad categories:
- **Discrete events:** Occur at a specific point in time, like submitting a form or receiving a message.
- **Continuous events:** Represent ongoing data, like a stream of sensor readings or stock price updates.

Producers:

- Producers are components that detect changes or trigger events.
 - They generate events based on an internal or external state change and publish them into the system. Examples include devices, applications, and user interfaces.

Consumers:

- Consumers are components that subscribe to and respond to events.
 - These components execute certain actions based on the event data, which could be logging the event, triggering additional services, or updating a user interface.

EDA Brokers and Communications

Event Brokers:

- Event brokers (or event buses) are intermediaries responsible for managing the routing of events between producers and consumers.
- They ensure the decoupling of producers and consumers, which allows for more scalable and flexible architectures.
- Popular brokers include **Apache Kafka**, **RabbitMQ**, and **AWS EventBridge**.

Synchronous vs. Asynchronous Communication:

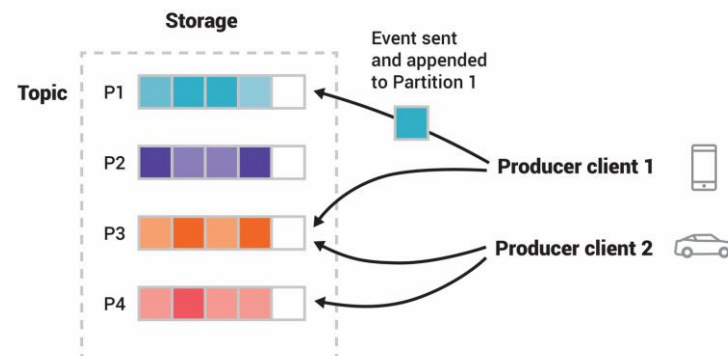
- **Synchronous:** In synchronous event handling, components communicate directly and must wait for a response (e.g., a client-server model).
- **Asynchronous:** In asynchronous event handling, event producers emit events without waiting for a response, and consumers process them independently.

MODERN EIP PLATFORMS

Apache Kafka



- Distributed event streaming platform for high-throughput, durable logs
- Core: topics ▶ partitions ▶ brokers; producers/consumers; consumer groups
- Storage: append-only commit log, retention & compaction; per-partition ordering
- Processing: Kafka Streams & ksqlDB for stateless/stateful stream processing
- Ops: KRaft mode (no ZooKeeper); schema mgmt via registry (Avro/JSON Schema/Protobuf)
- Strengths: massive throughput, strong horizontal scale, exactly-once processing with idempotence + transactions
- Common uses: event-driven microservices, CDC (Debezium), log/metric pipelines, real-time analytics



More than **80% of all Fortune 100 companies** trust, and use Kafka.

<https://kafka.apache.org/>

Apache Pulsar



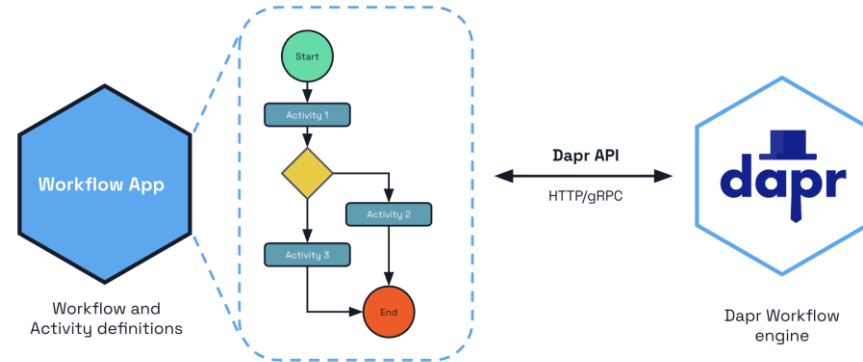
- Cloud-native messaging & streaming with compute/storage separation
- Core: brokers for routing; Apache BookKeeper for durable, segment-based storage
- Multi-tenancy, namespaces, geo-replication; tiered storage for cold data
- Subscription types: Exclusive, Failover, Shared, Key_Shared (per-key ordering + fan-out)
- Serverless processing: Pulsar Functions; rich IO connectors for sources/sinks
- Strengths: strong multi-tenant isolation, per-key fan-out, built-in DLQ & delayed redelivery
- Common uses: event meshes across teams, streaming + queue workloads in one platform

<https://pulsar.apache.org/>

Dapr



- Distributed Application Runtime (sidecar) that provides portable building blocks
- Building blocks: Service Invocation, Pub/Sub, Bindings, State, Actors, Secrets, Config, Observability
- Component model: plug-and-play backends (Kafka/Pulsar/RabbitMQ, Redis/SQL/NoSQL, cloud services)
- Contracts & conventions: CloudEvents for pub/sub; consistent APIs across languages
- Runs self-hosted or on Kubernetes; integrates with service meshes and OTEL
- Strengths: portability, polyglot support, faster path to production with standardized primitives
- Common uses: microservices with mixed stacks, cloud/vendor portability, gradual modernization



Temporal



Designing a Workflow engine from first principles

- Durable workflow orchestration engine for long-running, reliable business processes
- Key concepts: Workflows (deterministic), Activities (side-effecting), Task Queues, Signals & Queries
- Strong guarantees: state persisted as event history; automatic retries, backoff, timeouts, cron
- Execution model: application workers host workflows/activities via SDKs (Go/Java/TypeScript/Python)
- Change management: workflow versioning, patching, and migration without losing state
- Strengths: correctness under failure, human-in-the-loop and multi-day/month processes
- Common uses: payments/orders, provisioning, document processing, batch pipelines, SaaS automations

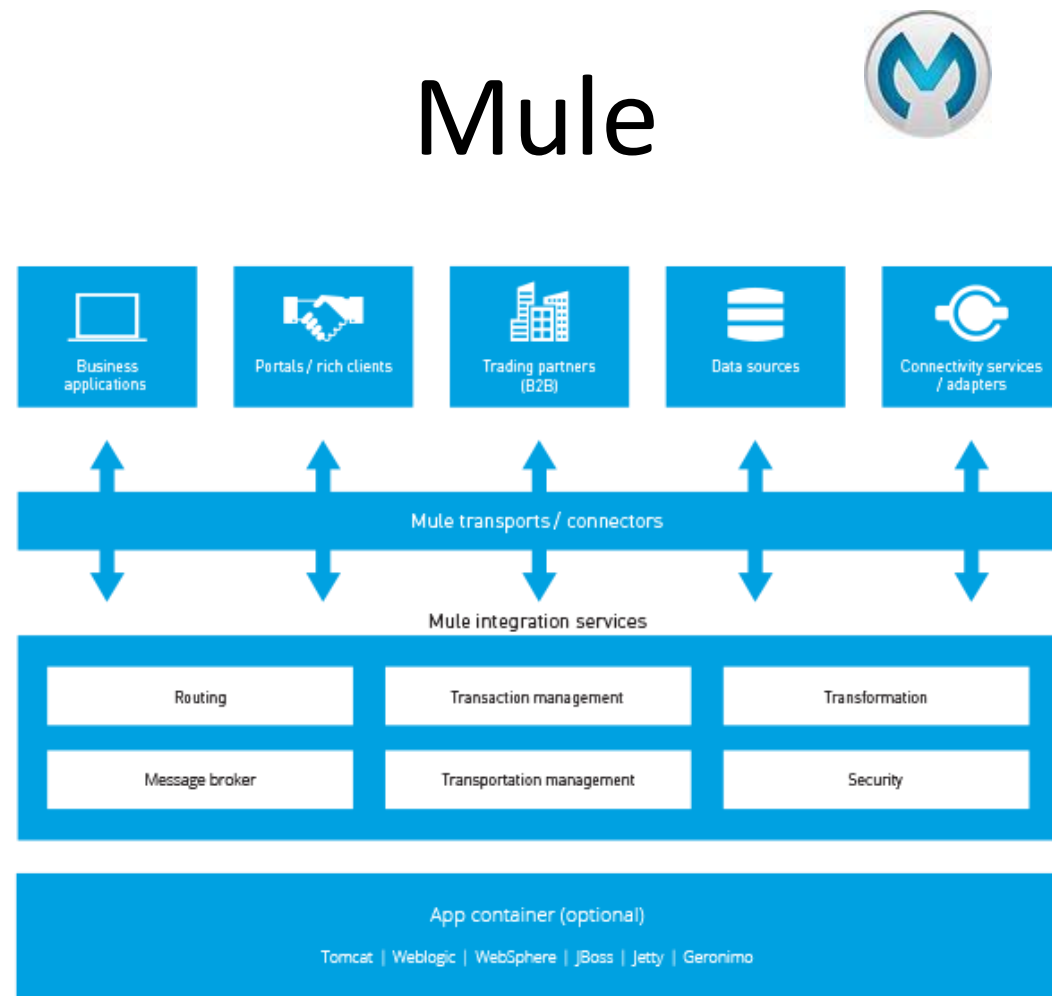
<https://temporal.io/blog/workflow-engine-principles>

EIP in 2026: Kafka, Pulsar, Dapr, Temporal

Pattern	Kafka	Pulsar	Dapr	Temporal
Pub/Sub	Topics; Consumer Groups	Topics; Shared/Key_Shared subs	Pub/Sub building block (broker-agnostic)	Signals (not a broker)
Message Channel	Topic per stream; partitions	Topic per stream; tenancy; geo-rep	Topics via component; CloudEvents	Task Queues (workflow <-> workers)
Competing Consumers	Consumer group rebalancing	Shared subscription; Key_Shared	Multiple subscribers behind topic	Multiple workers per task queue
Routing	Keys/Partitions; Kafka Streams/ksqlDB	Keys; Functions; topics by key	Message routing rules in pub/sub	Workflow logic routes activities
DLQ & Retries	DLQ topics; backoff; EOS/transactions	Built-in DLQ policy; delayed msgs	DLQ + backoff via component metadata	Automatic retries; backoff; compensation
Request/Reply	Reply topic + correlation ID	Async request/response APIs	Service Invocation (HTTP/gRPC)	Workflow result/queries (not pub/sub)
Idempotency	Idempotent producer; transactional writes	Message IDs; dedup; transactions	State store for dedup keys	Deterministic workflows; activity idempotency
Outbox/CDC	Debezium → Kafka outbox	CDC connectors → Pulsar	Bindings to DB/brokers; state	Not applicable (use with broker/DB)
Saga / Process Manager	Implement with streams/processors	Implement with Functions	Use state + pub/sub + bindings	First-class: workflows + activities + compensation

Mule is a lightweight enterprise service bus (**ESB**) and integration **framework**. The platform is Java-based, but can broker interactions between other platforms such as .NET using web services or sockets.

- Mule ESB is a lightweight Java-based enterprise service bus (ESB) and integration platform that allows developers to connect applications together quickly and easily, enabling them to exchange data.
- Easy integration of existing systems, including JMS, Web Services, JDBC, HTTP, and more.
- **Service creation and hosting** — expose and host reusable services, using Mule ESB as a lightweight service container
- **Service mediation** — shield services from message formats and protocols, separate business logic from messaging, and enable location-independent service calls
- **Message routing** — route, filter, aggregate, and re-sequence messages based on content and rules
- **Data transformation** — exchange data across varying formats and transport protocols



[What Mulesoft Does?](#)



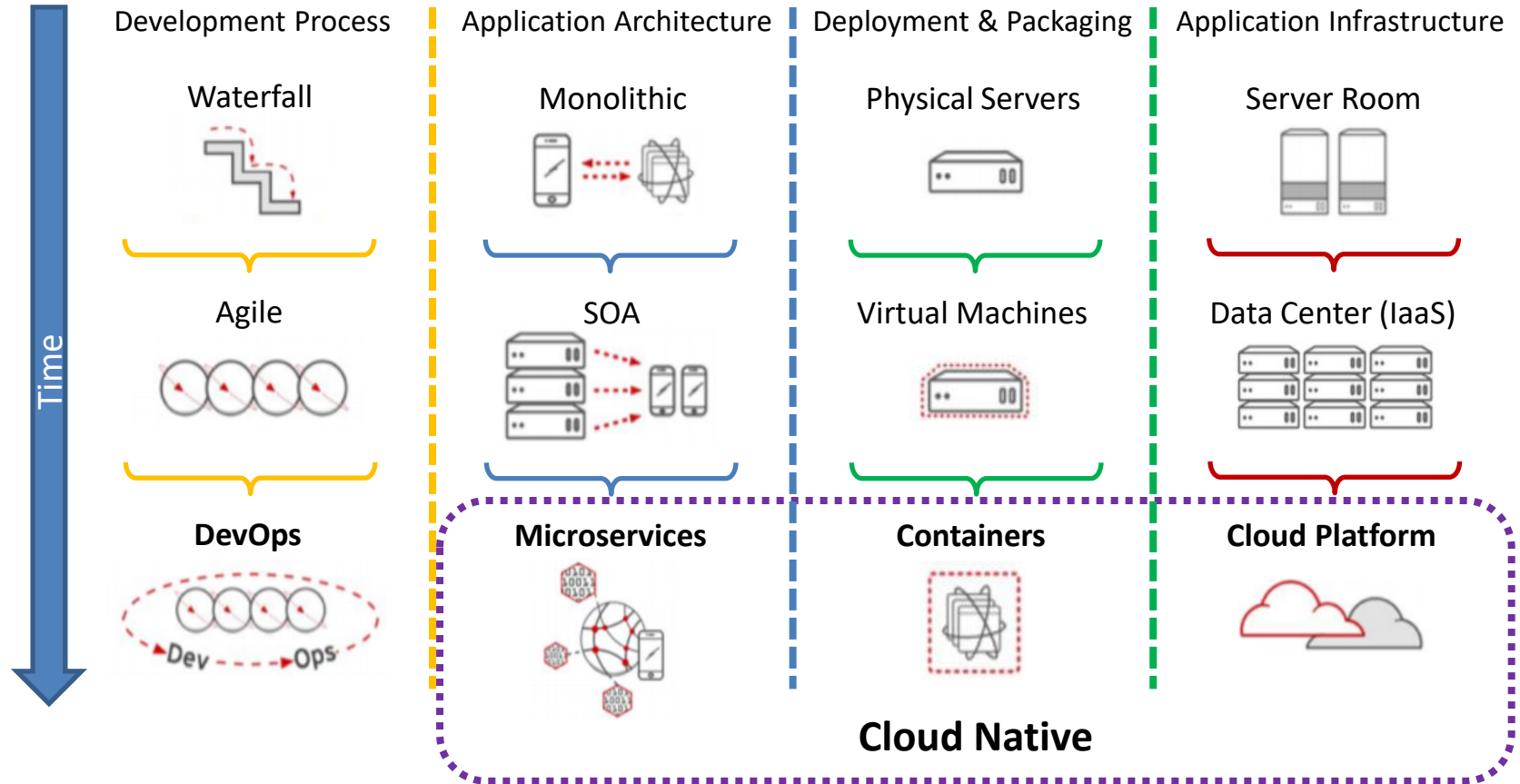
Starring
MARK HAMILL HARRISON FORD CARRIE FISHER
PETER CUSHING and ALEC GUINNESS
Written and Directed by GEORGE LUCAS
Produced by GARY KURTZ
Music by JOHN WILLIAMS
Directed by PAUL SHERMAN and JOHN COOPER

HILDEBRANDT

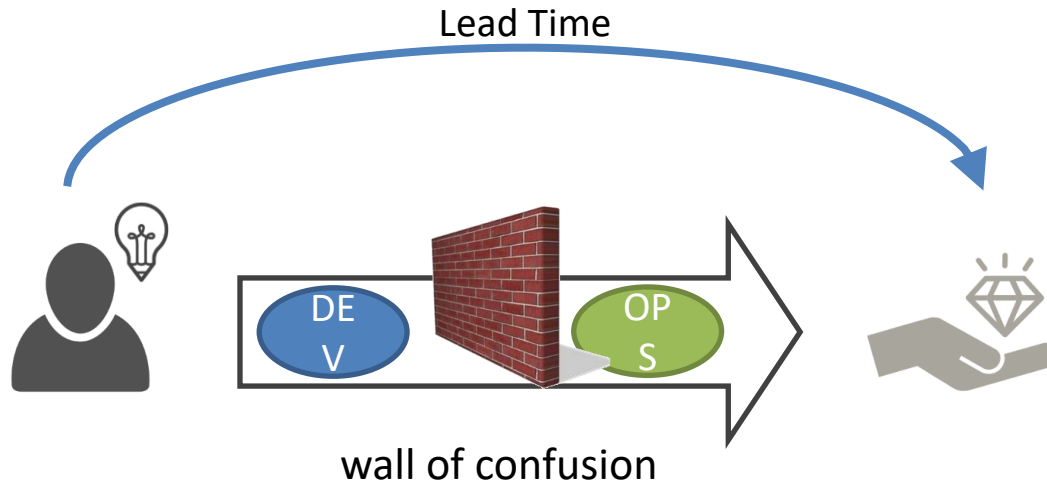
A NEW HOPE

after a short break..

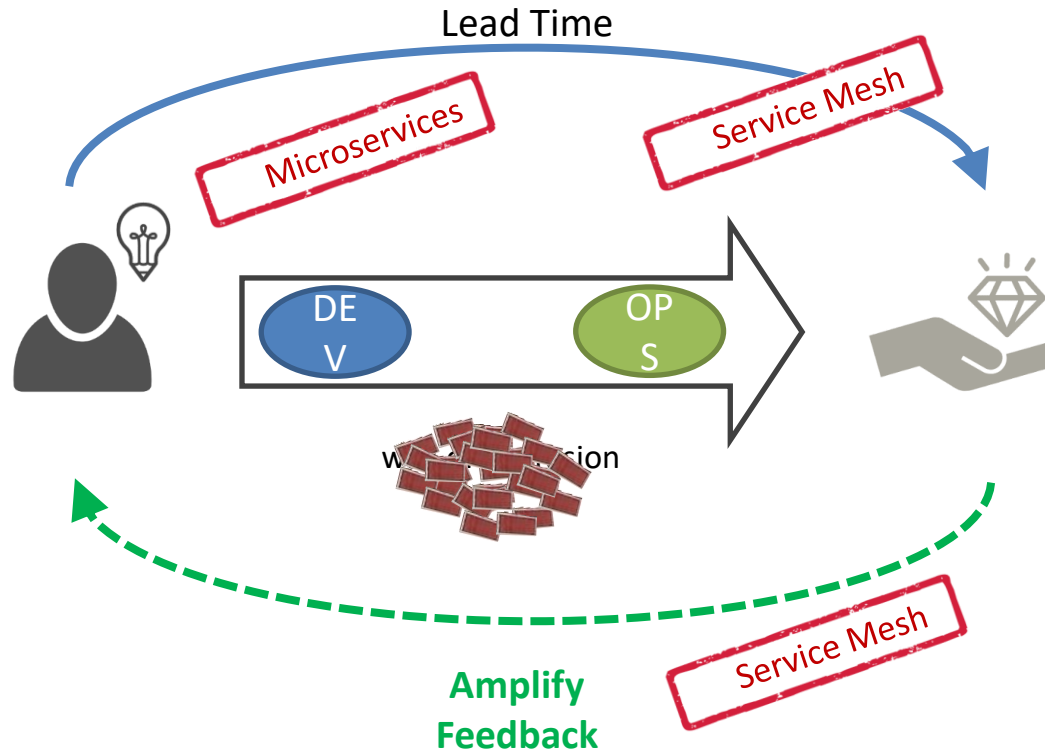
The Way Applications Are Developed and Deployed Has Fundamentally Changed



Introduction – DevOps Drivers

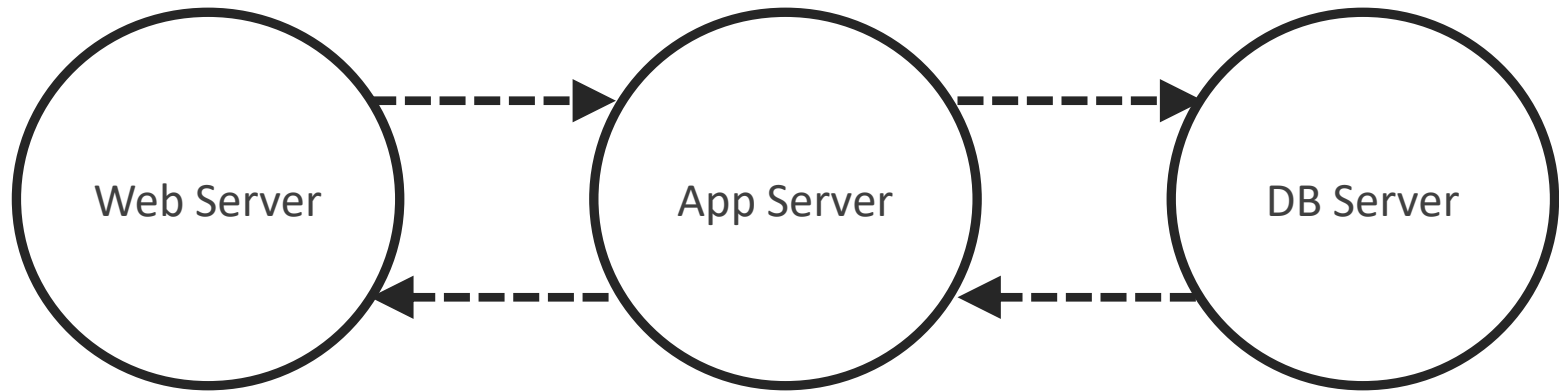


Introduction – DevOps Drivers

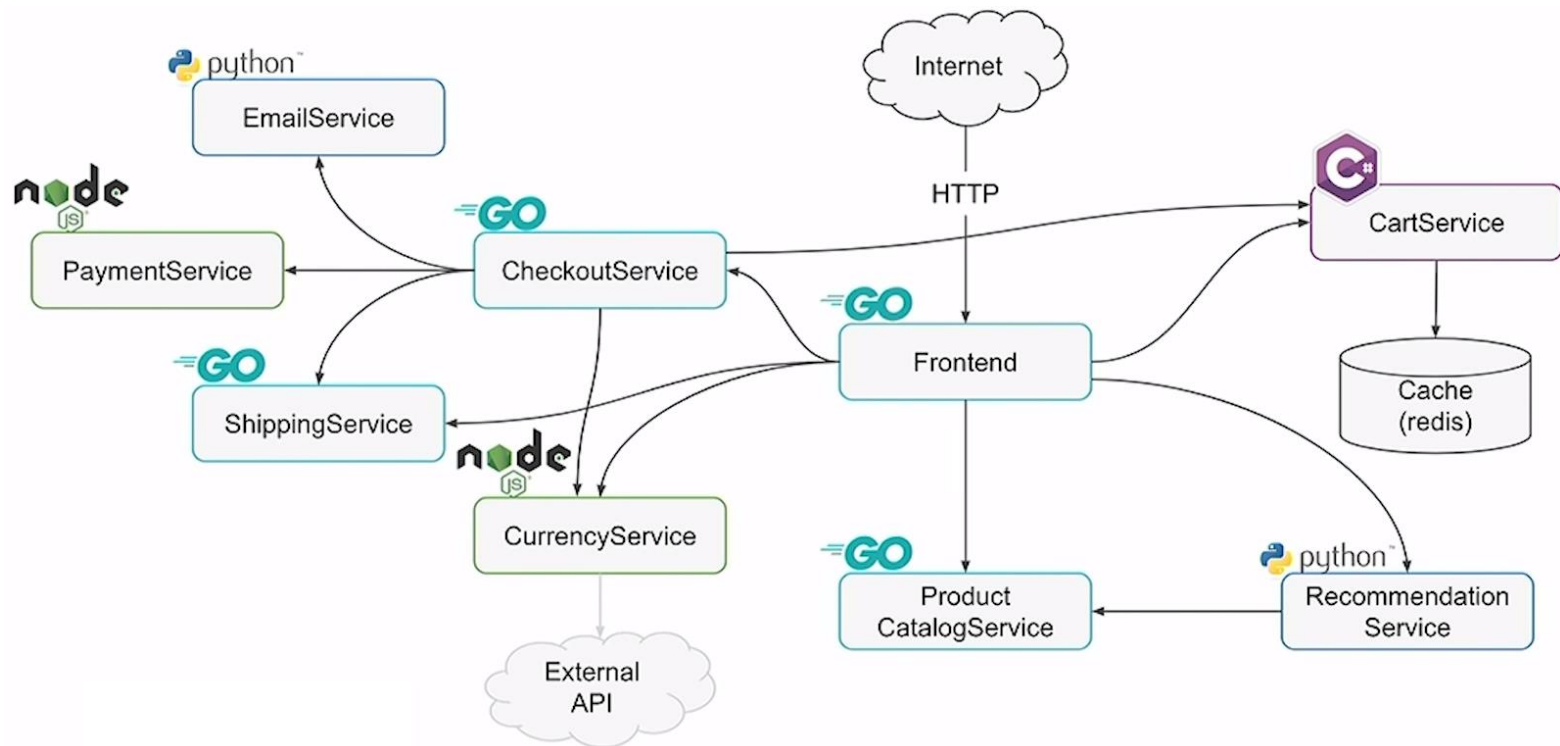


**What do Microservices
typically look like?**

This is not a Microservice Architecture!

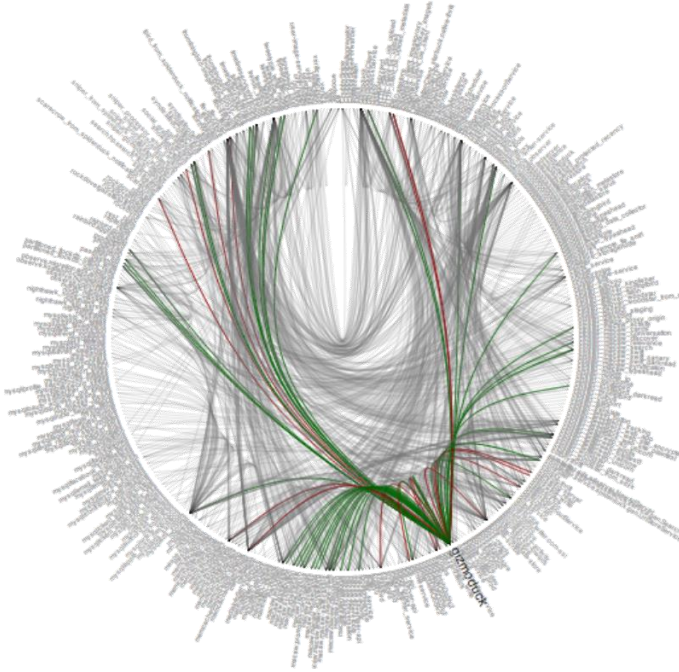


This is Getting There....

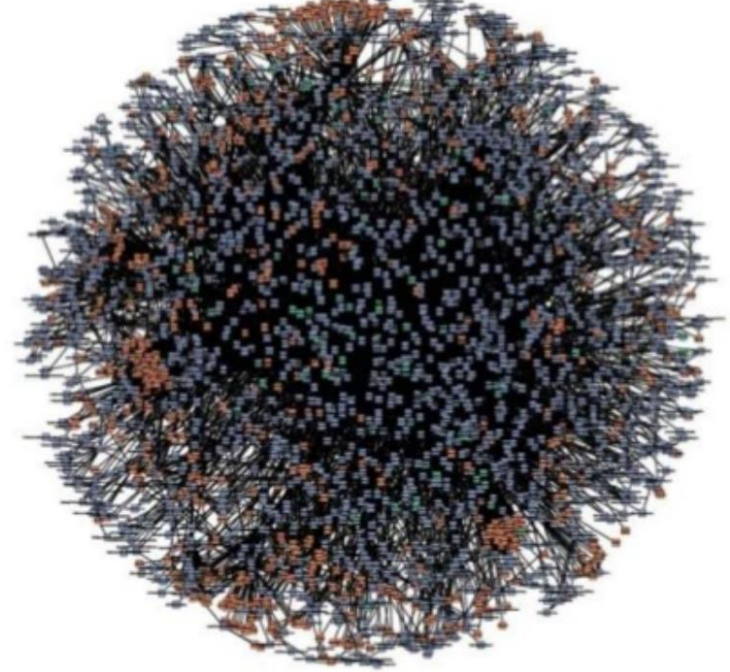


But These are True Microservice Architectures!

Twitter



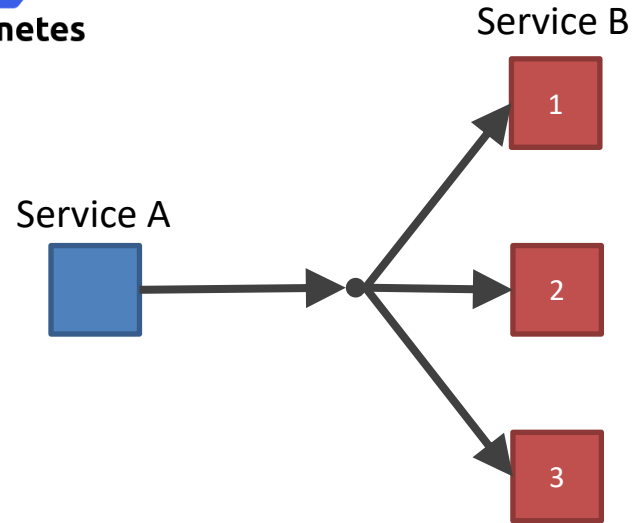
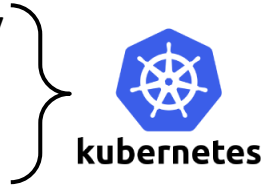
Amazon Web Service



Complexity moved from application into the network!

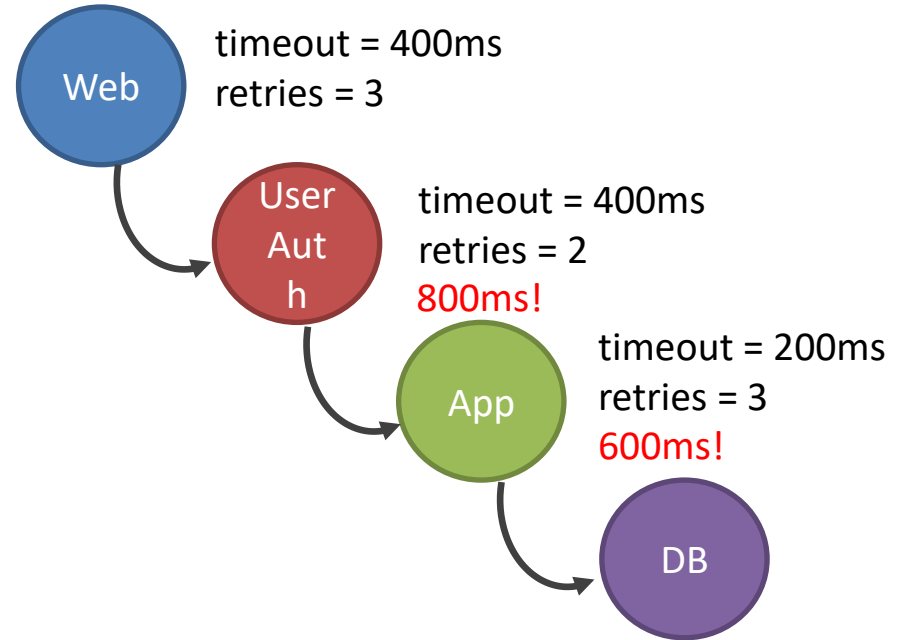
The Need of Common Features in Service Components

- Dynamic service discovery
- Load balancing
- Health checks



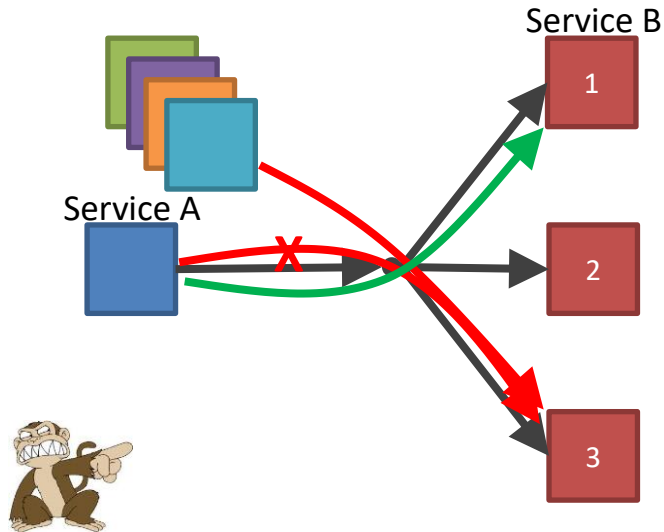
The Need of Common Features in Service Components

- Dynamic service discovery
- Load balancing
- Health checks
- Timeouts
- Retries



The Need of Common Features in Service Components

- Dynamic service discovery
- Robust load balancing algorithms
- Health checks
- Cascading failure prevention (circuit breaking)
- Control over request routing (useful for things like CI/CD release patterns)
- Resiliency features (retries, timeouts, deadlines, etc)
- The ability to introduce and manage TLS termination between communication endpoints
- Monitoring and tracing via rich metrics
- Fault injection (Chaos Monkey)



Making the Netflix API More Resilient



Netflix Technology Blog [Follow](#)
Dec 8, 2011 · 7 min read

by Ben Schmaus

The API brokers catalog and subscriber metadata between internal services and Netflix applications on hundreds of device types. If any of these internal services fail there is a risk that the failure could propagate to the API and break the user experience for members.

The Canonical Example: NETFLIX



NETFLIX

Mastering Chaos - A Netflix Guide to Microservices

- <https://www.youtube.com/watch?v=CZ3wluvmHeM>

What is a Service Mesh?

Service Mesh is **dedicated infrastructure layer** in a **microservice environment** to consistently **manage, monitor** and **control** the **communication** between services across the entire application

The Service Mesh Pattern

The service mesh pattern is focusing on managing all service-to-service communication within a distributed software system.

Context

The context for the pattern is twofold:

- First, that engineers have adopted the microservice architecture pattern, and are building their applications by composing multiple (ideally single-purpose and independently deployable) services together
- Second, the organizations have embraced cloud native platform technologies such as containers (e.g., Docker), orchestrators (e.g., Kubernetes), and gateways

Service Mesh: Intent

The problems that the service mesh pattern attempts to solve include:

- Eliminating the need to compile into individual services a language-specific communication library to handle service discovery, routings, and application-level (Layer 7) non-functional communication requirements.
- Externalizing service communication configuration, including network locations of external services, security credentials, and quality of service targets.
- Providing passive and active monitoring of other services.
- Decentralizing the enforcement of policy throughout a distributed system.
- Providing observability defaults and standardizing the collection of associated data.
 - Enabling request logging
 - Configuring distributed tracing
 - Collecting metrics

Service Mesh Features

- Normalizes naming and adds logical routing, (e.g., maps the code-level name “user-service” to the platform-specific location “AWS-us-east-1a/prod/users/v4”)
- Provides traffic shaping and traffic shifting
- Maintains load balancing, typically with configurable algorithms
- Provides service release control (e.g., canary releasing and traffic splitting)
- Offers per-request routing (e.g., traffic shadowing, fault injection, and debug re-routing)
- Adds baseline reliability, such as health checks, timeouts/deadlines, circuit breaking, and retry (budgets)
- Increases security, via transparent mutual Transport Level Security (TLS) and policies such as Access Control Lists (ACLs)
- Provides additional observability and monitoring, such as top-line metrics (request volume, success rates, and latencies), support for distributed tracing, and the ability to “tap” and inspect real-time service-to-service communication
- Enables platform teams to configure “sane defaults” to protect the system from bad communication

Service mesh capabilities

- Connectivity
- Reliability
- Security
- Observability

SERVICE MESH EVOLUTION

Evolution: 1G - Shared Libraries

Examples for such fat libraries:

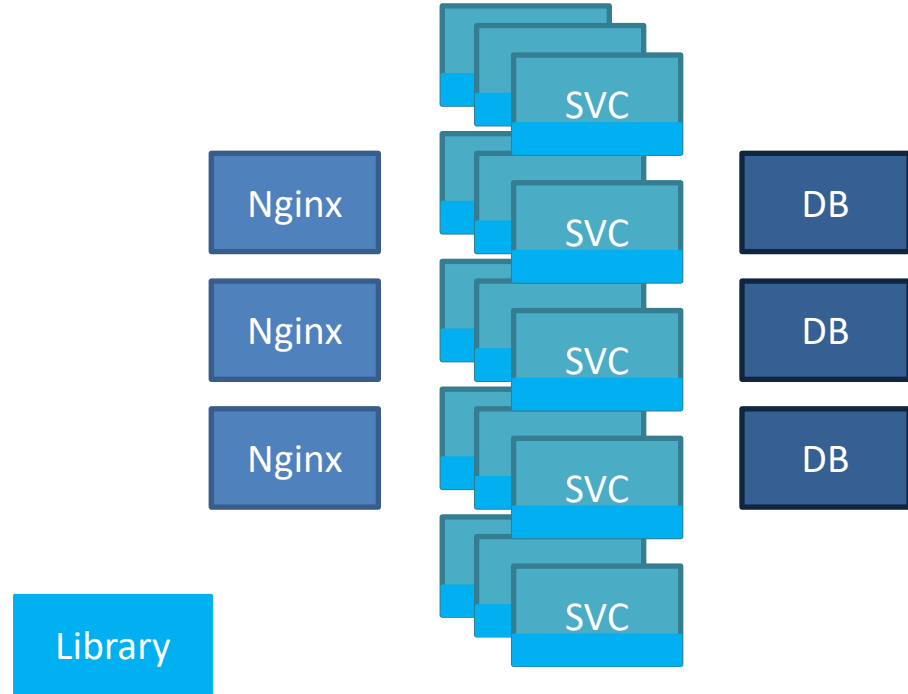
- Hystrix @ Netflix
- Stubby @ Google
- Finagle @ Twitter

Advantages:

- Simple
- Customizable

Disadvantages of shared libraries:

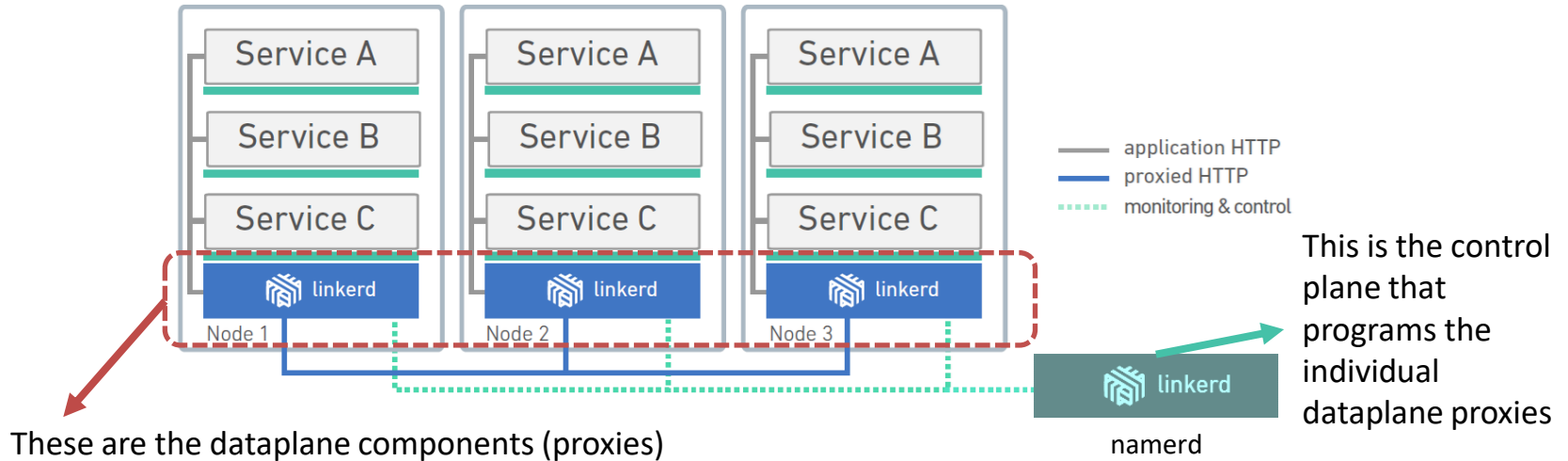
- Have to be implemented in multiple languages
- If the library changes the entire service has to be redeployed
- Too tight involvement of dev teams



Evolution: 2G - Linkerd

A service mesh that adds reliability, security, and visibility to cloud native applications

- Official CNCF Project
- Originally created by Buoyant Inc. based on Finagle
- Written in JAVA



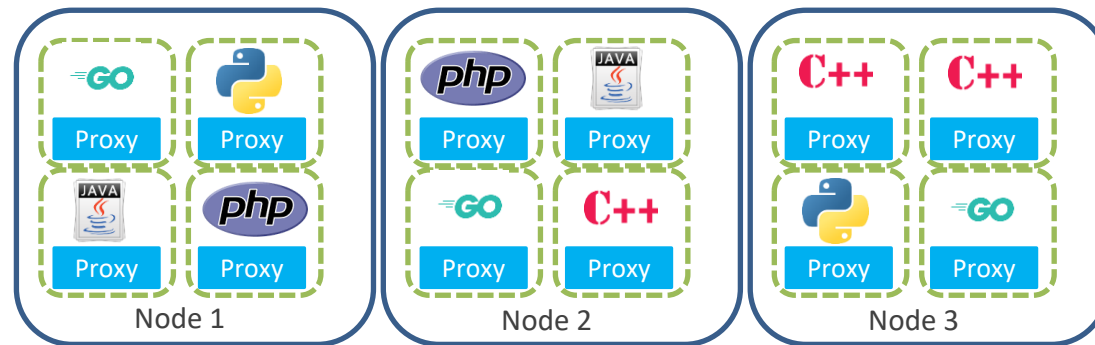
Per-node vs. Sidecar Model

The per-node model and its disadvantages:

- Raises security concerns in multi-tenant environments (shared TLS secrets, etc.)
- Can only be scaled vertically, not horizontally (more memory and CPU handle more conns.)
- Not optimized for container workloads

The sidecar model:

- Put a proxy next to every container
- This is supported by the POD abstraction in Kubernetes
- Linkerd is considered to be too heavy for such environment!



Evolution: 3G - Service Meshes

- Linkerd2 (Conduit):

- Specifically designed for Kubernetes: control plane is written in Go
- Data plane is written in Rust: fast and lightweight to sidecar operations (~5MB container size)
- Can be deployed service-by-service (it's not an all-or-nothing choice...)



- Envoy

- Open source edge and service proxy, designed for cloud-native applications
- Official CNCF Project, originally created by Lyft
- Written in C++, built on the learnings of solutions as NGINX, HAProxy, hardware LBs and cloud LBs

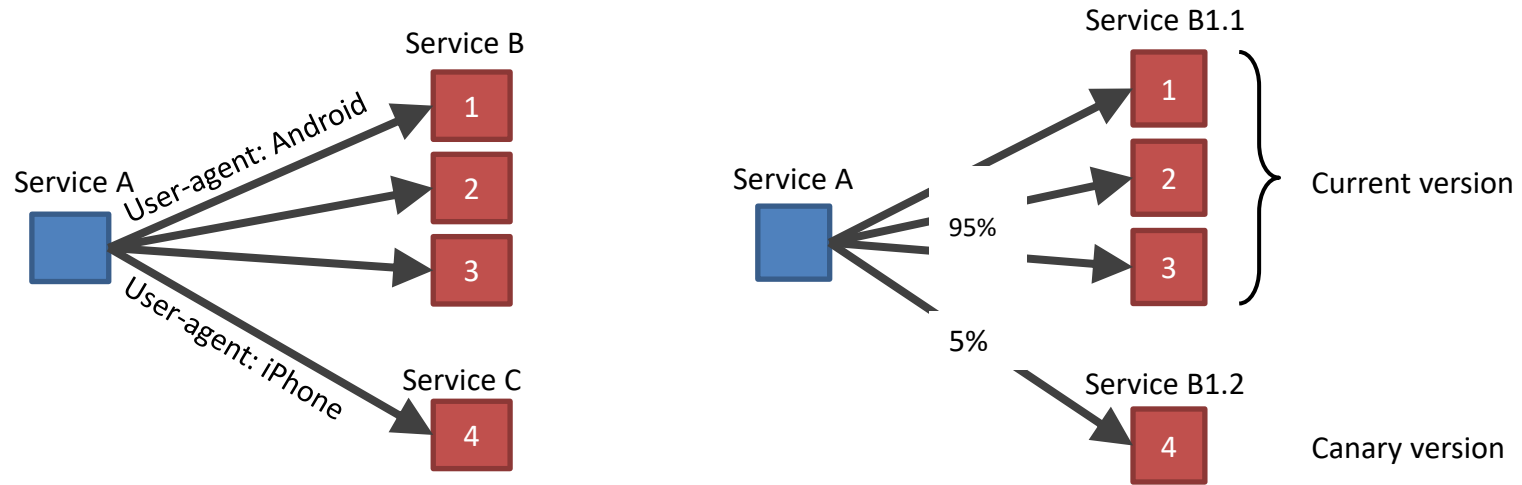


- Istio

- Service mesh control plane which uses Envoy as data plane
- Originally created by Google and IBM
- Written in GO



Examples: Routing



TCP 8080

/api/v1/t-shirts

GET



1000/day

/api/v1/t-shirts

PUT

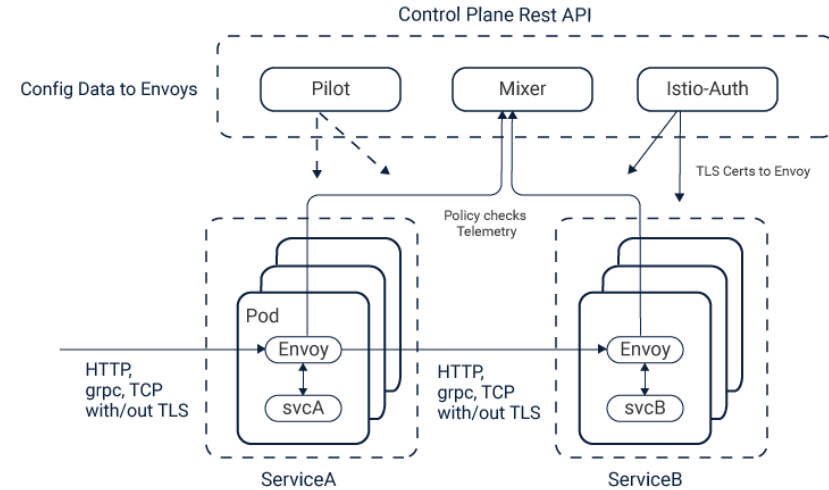
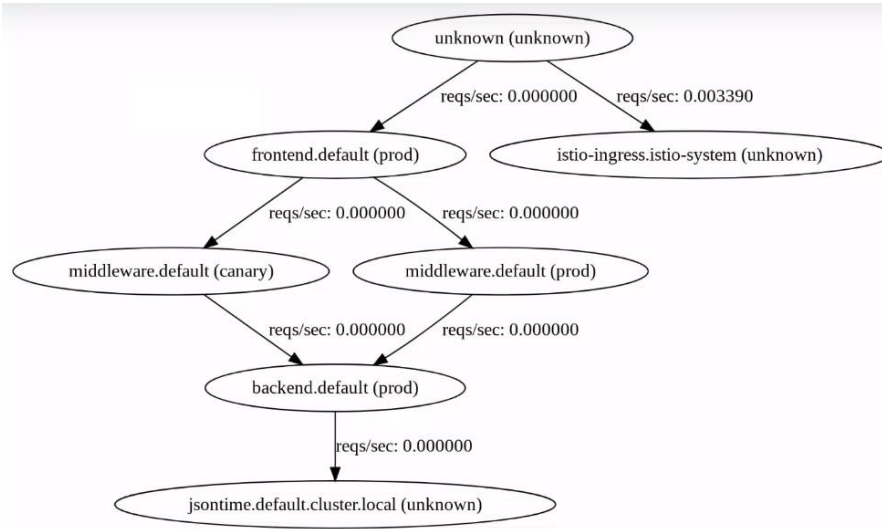


/api/v1/jeans

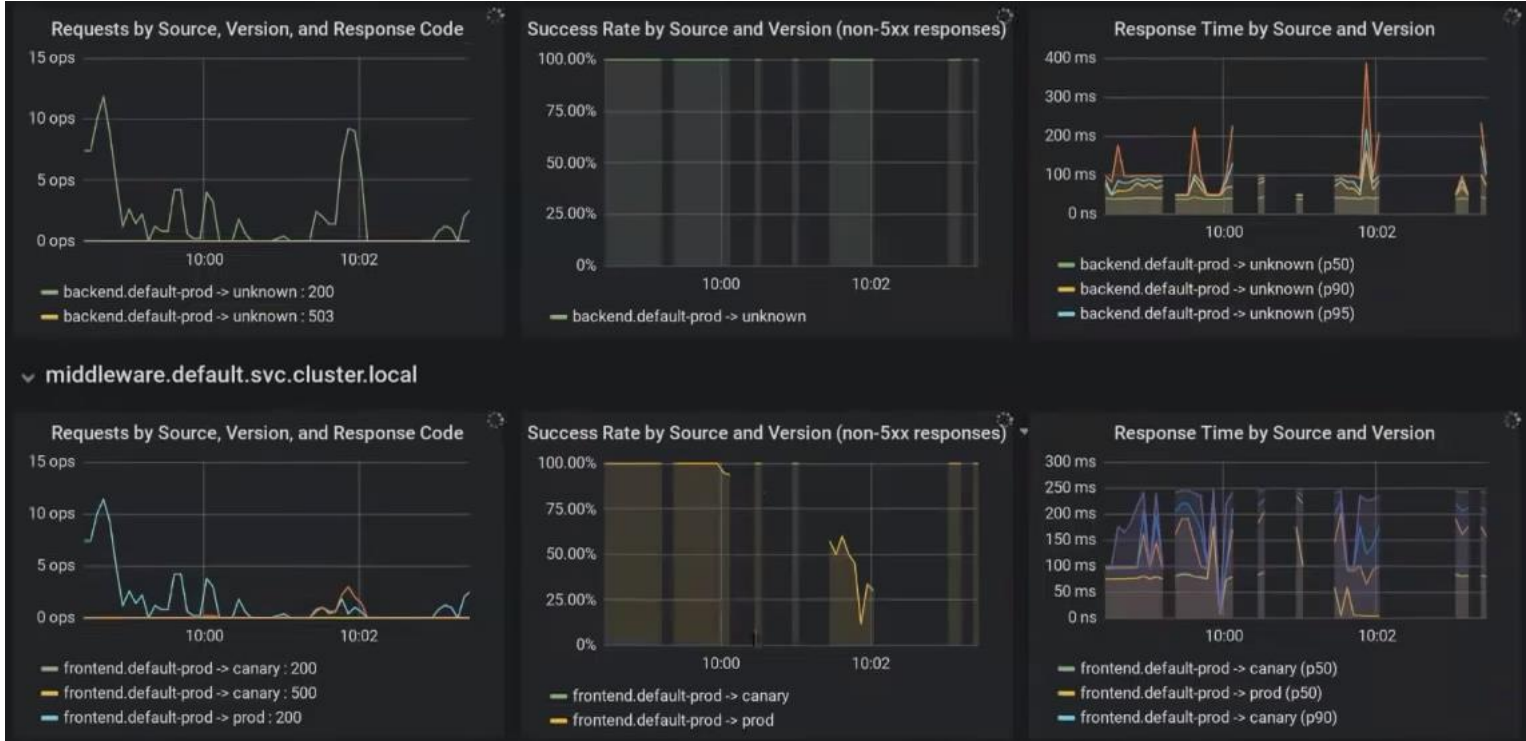
*



Examples: Visibility (Istio Pilot)



Examples: Visibility



Comparison

3	Name	Linkerd	Istio/Envoy	Nginx Plus	
4	<i>Website</i>	https://linkerd.io/	https://istio.io/	https://www.nginx.com/products/	
5	<i>Licence</i>	AL2.0	AL2.0	Commercial	
6	<i>Custodians</i>	Buoyant (Twitter pedigree) Buoyant	Lyft, Google, IBM	Nginx Inc	
7	<i>Build language</i>	Scala/Rust	Go/C++11	C	
8	<i>Deployment platforms</i>	Any	Kubernetes	Any	
9	<i>Protocols</i>	HTTP/2, gRPC, TCP	HTTP/2, gRPC, TCP	HTTP/1.1, TCP, UDP	
10	<i>Service discovery integration</i>	File, Consul, ZK, etcd, K8s, Marathon	No (adapters in future)	File, Consul, ZK, etcd, K8s, Marathon (nixy)	
11	<i>Container Deployment</i>	Per Host/Sidecar	Sidecar	Per Host/Sidecar	
12	<i>Request Tracing</i>	Yes (Zipkin)	Yes (Zipkin)	Possible, but bespoke	
13	<i>Plugin language</i>	Java plugins	Golang plugins	C modules	
14	<i>Commercial support</i>	Yes (Buoyant)	No?	Yes (Nginx Inc)	
15	<i>Production Deploys</i>	PayPal, Expedia, AOL, Monzo	Lyft (Envoy)	WIX, ARM, Bluestem	
16					

Service Mesh Standards

- [Service Mesh Interface](#) (SMI). The Service Mesh Interface is a specification for service meshes that run on [Kubernetes](#). It doesn't implement a service mesh itself but defines a common standard that can be implemented by a variety of service mesh providers
 - SMI is basically a collection of Kubernetes Custom Resource Definitions ([CRD](#)) and Extension API Servers
 - These APIs can be installed onto any Kubernetes cluster and manipulated using standard tools
 - To activate these APIs, an SMI provider is run in the Kubernetes cluster
- API specifications include the following:
 - Traffic Access Control
 - Traffic Metrics
 - Traffic Specs
 - Traffic Split

Service Mesh Tutorials

- [Layer 5 Meshery](#)—a multi-service mesh management plane
- [Solo's Gloo Mesh](#)—a service mesh orchestration platform
- [KataCoda Istio tutorial](#)
- [Consul service mesh tutorial](#)
- [Linkerd tutorial](#)
- [NGINX Service Mesh Tutorial](#)

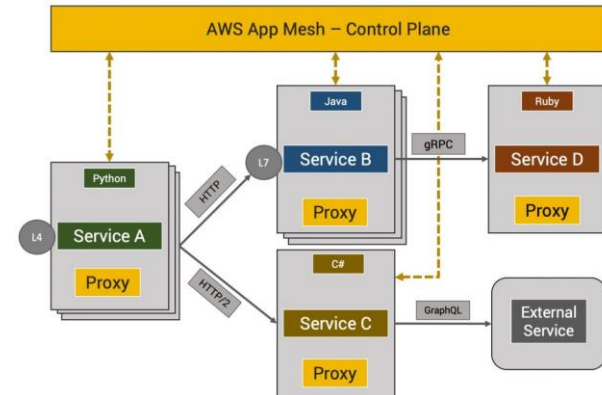
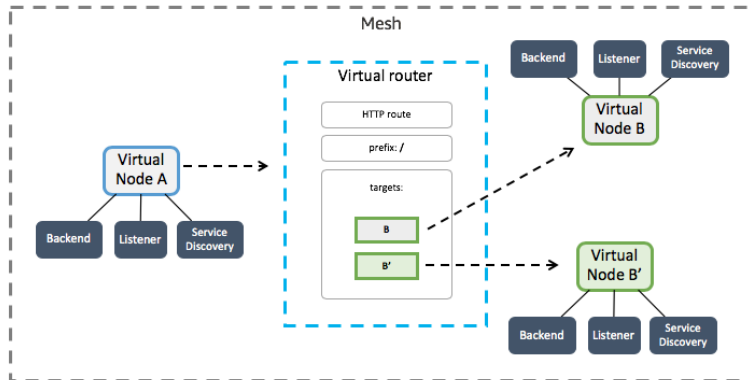
AWS App Mesh



- [AWS App Mesh](#) is a service mesh that makes it easy to monitor and control services
- An infrastructure layer dedicated to handling service-to-service communication, through an array of lightweight network proxies deployed alongside the application code
- App Mesh standardizes how services communicate, giving end-to-end visibility and ensuring high availability
- App Mesh gives consistent visibility and network traffic controls for every service in an application

Components of AWS App Mesh

- **Service mesh** – A service mesh is a logical boundary for network traffic between the services that reside within it. In the example, the mesh is named apps, and it contains all other resources for the mesh. For more information, see Service Meshes.
- **Virtual services** – A virtual service is an abstraction of an actual service that is provided by a virtual node, directly or indirectly, by means of a virtual router. In the illustration, two virtual services represent the two actual services
- **Virtual nodes** – A virtual node acts as a logical pointer to a discoverable service, such as an Amazon ECS or Kubernetes service. For each virtual service, you will have at least one virtual node.
- **Virtual routers and routes** – Virtual routers handle traffic for one or more virtual services within your mesh. A route is associated to a virtual router. The route is used to match requests for the virtual router and to distribute traffic to its associated virtual nodes.
- **Proxy** – You configure your services to use the proxy after you create your mesh and its resources. The proxy reads the App Mesh configuration and directs traffic appropriately.



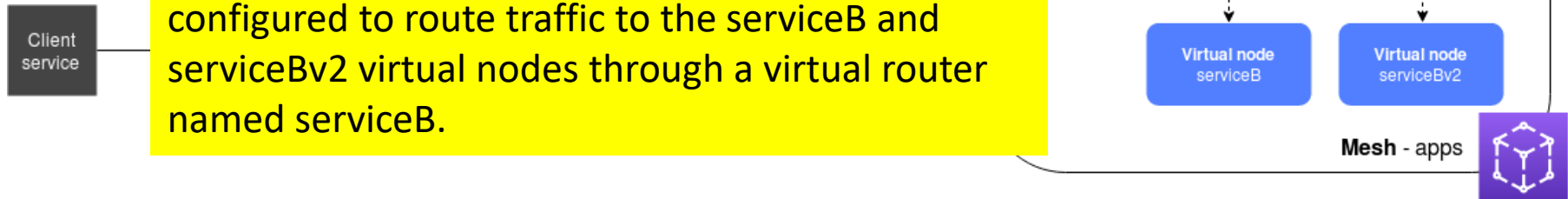
Adding App Mesh to an example application

- Consider example App, that doesn't use App Mesh.
- E.g., AWS Fargate, Amazon Elastic Container Service (Amazon ECS), Amazon Elastic Kubernetes Service (Amazon EKS), Kubernetes on Amazon

Ela
ins
wit

In the illustration, the servicea.apps.local virtual service gets configuration information for the virtual node named serviceA.

The serviceA virtual node is configured with the servicea.apps.local name for service discovery. The serviceb.apps.local virtual service is configured to route traffic to the serviceB and serviceBv2 virtual nodes through a virtual router named serviceB.



What's changed since the 2010s

- From monoliths & ESBs → cloud-native, API-first, event-driven
- HTTP/3 + QUIC; GraphQL federation; gRPC/Connect for internal APIs
- Edge/serverless runtimes (Workers, Functions) alongside containers
- Service mesh shift: sidecars → sidecar-less (ambient), eBPF-powered data planes
- Unified observability with OpenTelemetry (traces, metrics, logs)

Modern protocols & API design

- REST still dominant; OpenAPI 3.1 for contracts
- GraphQL with schema federation (Apollo Federation v2)
- gRPC (Protobuf) for low-latency, strongly-typed internal services
- Connect RPC (Buf) & tRPC (TypeScript) for dev-velocity in JS/TS stacks
- AsyncAPI for event streams & async messaging

Server-side frameworks (by ecosystem)

- **JVM:** Spring Boot 3.x (AOT, native images), Quarkus 3, Micronaut 4
- **.NET:** ASP.NET Core on .NET 9, Minimal APIs
- **Node.js/TS:** NestJS 11, Fastify; Deno/Bun runtimes where appropriate
- **Python:** FastAPI for typed, async web APIs
- **Go/Rust:** Gin/Fiber (Go), Axum/Actix (Rust) for high-perf services

Frontend frameworks & rendering patterns

- React 19 + Server Components; Next.js App Router for hybrid SSR/SSG/streaming
- SvelteKit 2, Angular 18/19 (signals), Qwik (resumability), Vue 3
- Islands architecture, progressive enhancement, htmx for hypermedia UIs

Integration & workflow platforms

- Event streaming: Apache Kafka, Apache Pulsar for durable logs/queues
- Dapr building blocks: service invocation, pub/sub, bindings, actors, state
- Temporal (durable execution) for long-running, reliable workflows

Kubernetes networking & service meshes

- Gateway API as a standard L4/L7 entry to clusters (supersedes Ingress)
- Istio Ambient Mesh (sidecar-less) for L4/L7; mTLS, traffic policy, telemetry
- Cilium Service Mesh using eBPF for dataplane efficiency

Observability & reliability

- OpenTelemetry everywhere: SDKs + Collector for traces/metrics/logs
- SLOs, RED/USE methods; tracing driven performance debugging
- Resilience patterns: timeouts, retries, circuit breakers, bulkheads

Security & policy

- OAuth 2.1 / OIDC for authN/Z
- mTLS in mesh
- Zero-Trust networking
- OPA/Gatekeeper for policy-as-code;
OpenFeature for feature flags

MODERN API INTEGRATION

Contract-First API Design — Why it matters

- **Design the API contract first;** code follows the spec (product, not by-product).
- **Single source of truth:** endpoints, schemas, errors, auth, examples.
- **Benefits:** alignment across teams, parallel dev with mocks, fewer reworks, governance.
- **Choose the right DSL:** **OpenAPI** for HTTP APIs; **AsyncAPI** for evented interfaces.

OpenAPI 3.1 — Essentials for a solid contract

- Full JSON Schema compatibility (2020-12)
- Paths & operations with clear HTTP semantics
- components/: reusable schemas, parameters, headers
- Servers with variables; tags for grouping; webhooks/callbacks when needed.
- Ship openapi.yaml in repo root; keep examples realistic and testable.

Modeling resources & using HTTP correctly

- Model nouns, not verbs: /issues, /issues/{number}, /issues/{number}/comments.
- Status codes: 201 + Location on create; 200/204 on update; 4xx for client errors.
- Pagination: page & per_page; propagate Link headers; filtering & sorting rules.
- Idempotency: keys for POST when needed; safe PATCH with ETag + If-Match.
- Errors: consistent Problem Details object (RFC 9457); include correlation id.

Versioning & schema evolution without breaking clients

- Prefer additive changes: new optional fields; do not repurpose semantics.
- Deprecate in schema; publish a change log; communicate Sunset dates.
- Versioning: header-based or URL prefix — be consistent and sparing.
- CI compatibility checks to prevent accidental breaking changes.

Design → Mock → Generate → Test → Ship (toolchain)

- Author & lint: VS Code + Spectral; style guide rules (naming, status codes, errors).
- Mock: run a realistic mock with Prism from the OpenAPI doc for parallel dev.
- Generate: clients/servers/docs via OpenAPI Generator in CI; publish SDKs.
- Test: contract tests (Dredd) + property-based tests (Schemathesis).
- Automate: validate spec in CI; publish to an internal registry/portal.

Homework tie-in — Issues Gateway contract

- Define `/issues`, `/issues/{number}`, `/issues/{number}/comments` with request/response schemas.
- Security: `bearerAuth` scheme; document scopes; require proper `Accept` header.
- Pagination & errors: Link headers, Problem Details schema, example responses (200/201/4xx).
- Quality gates: Spectral rules, Schemathesis, Dredd happy-path checks in CI.
- Deliverables: `openapi.yaml` (3.1), mock server script, generated client for tests.

Example OpenAPI Contract

```
openapi: 3.1.0
info: {title: Issues API (demo), version: 1.0.0}
servers: [{url: https://api.example.com}]
security: [{ bearerAuth: [] }]
paths:
  /issues:
    get:
      parameters:
        - {name: state, in: query, schema: {type: string, enum: [open, closed, all], default: open}}
        - {name: labels, in: query, style: form, explode: false, schema: {type: array, items: {type: string}}}
      responses:
        '200': {description: OK, content: {application/json:
          {schema: {type: array, items: {$ref: '#/components/schemas/Issue'}}}}}
    post:
      requestBody: {required: true, content: {application/json:
        {schema: {$ref: '#/components/schemas/NewIssue'}}}}
      responses:
        '201': {description: Created, headers: {Location: {schema: {type: string, format: uri}}},
          content: {application/json: {schema: {$ref: '#/components/schemas/Issue'}}}}
  /issues/{number}:
    get:
      parameters: [{name: number, in: path, required: true, schema: {type: integer, minimum: 1}}]
      responses: {'200': {description: OK, content: {application/json: {schema: {$ref: '#/components/schemas/Issue'}}}},
        '404': {description: Not found}}
components:
  securitySchemes: { bearerAuth: {type: http, scheme: bearer} }
  schemas:
    Issue: {type: object, required: [number, title], properties:
      {number: {type: integer}, title: {type: string}, state: {type: string, enum: [open, closed]}}}
    NewIssue: {type: object, required: [title], properties: {title: {type: string}, body: {type: string}}}
```

Swagger (OpenAPI Tooling)



- **What:** Swagger is the tooling ecosystem for OpenAPI (the spec).
- **Why:** generates interactive docs, mock servers, and client/server stubs.
- **Key tools:** Swagger UI (renders & "Try it out"), Swagger Editor (browser), Codegen / OpenAPI Generator.
- **How** (fastest): open <https://editor.swagger.io>, paste your OpenAPI YAML, and test endpoints.
- **Best practice:** commit openapi.yaml to your repo; host Swagger UI at /docs for students.
- **Terminology:** 'OpenAPI' = spec; 'Swagger' = tools (the spec used to be called Swagger).

Quic Demo

- 1) Go to <https://editor.swagger.io>
- 2) Paste your OpenAPI YAML
- 3) Click "Try it out" on an operation

```
# Serve Swagger UI with Docker (optional)
docker run -p 8080:8080 -e SWAGGER_JSON=/openapi.yaml \
  -v $(pwd)/openapi.yaml:/openapi.yaml swaggerapi/swagger-ui
```

IN-CLASS EXERCISE

In-Class Exercise - Amazon ECS Service

Connect with the AWS CLI

- <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/create-service-connect.html>

You can create an Amazon ECS service for a Fargate task that uses Service Connect with the AWS CLI.

The following are Service Connect prerequisites:

- Verify that the latest version of the AWS CLI is installed and configured. For more information, see [Installing or updating to the latest version of the AWS CLI](#).
- Your IAM user has the required permissions specified in the [AmazonECS_FullAccess](#) IAM policy example.
- You have a VPC, subnet, route table, and security group created to use. For more information, see [Create a virtual private cloud](#).
- You have a task execution role with the name `ecsTaskExecutionRole` and the `AmazonECSTaskExecutionRolePolicy` managed policy is attached to the role. This role allows Fargate to write the NGINX application logs and Service Connect proxy logs to Amazon CloudWatch Logs. For more information, see [Creating the task execution role](#).

Step 1: Create the cluster

Step 2: Create the service for the server

Step 3: Verify that you can connect



HOMEWORK

HW #2 – GitHub Service



Build a small service that wraps the GitHub REST API for Issues for a single repository you control.

Your service must:

- 1) Expose a clean HTTP API (your own endpoints) for issue CRUD* and comments,
- 2) Validate and process GitHub webhooks (issues + issue_comment),
- 3) Ship an OpenAPI 3.1 contract for your API,
- 4) Include automated tests (unit + integration), and
- 5) Provide a one-click/dev-container or Docker run script.

NOTES:

- Include in the source code comments, i.e. who wrote which code.
- Code should include Unit tests (e.g. junit, TestNG, unittest, etc..)
- Submit a Word Document of UI interaction, with screenshots

Good Coding.

References

- <http://www.soapatterns.org/>
- *Patterns of Enterprise Application Architecture By: Martin Fowler Publisher: Addison-Wesley Professional*
- <http://martinfowler.com/articles/enterprisePatterns.html>
- <http://pubs.opengroup.org/architecture/togaf8-doc/arch/chap28.html>
- <http://www2.it.lut.fi/wiki/lib/exe/fetch.php/courses/cs30a7400/i112.pdf>
- <http://pubs.opengroup.org/architecture/togaf9-doc/arch/>
- <http://www.forbes.com/sites/jasonbloomberg/2014/08/07/enterprise-architecture-dont-be-a-fool-with-a-tool/>
- <http://www.mikethearchitect.com/2013/02/togaf-demystification-series-comparing-togaf-to-other-frameworks.html>
- <https://www.manning.com/books/camel-in-action>
- <http://www.enterpriseintegrationpatterns.com/>
- http://www.opengroup.org/architecture/0210can/togaf8/doc-review/togaf8cr/c/p4/zf/zf_mapping.htm
- <https://enectoux.wordpress.com/2010/12/07/top-four-enterprise-architecture-methodologies/>
- <http://www.forbes.com/sites/joemckendrick/2012/09/18/before-there-was-cloud-computing-there-was-soa/>
- <http://www.webassist.com/tutorials/PayPal-Sandbox-for-testing>
- https://www.hwsu.hu/kepek/hirek/2019/04/hwsu0402/hwsu_devops_szabo.pptx
- <https://www.infoq.com/articles/service-mesh-ultimate-guide-2e/>
- <https://thenewstack.io/aws-app-mesh-amazons-own-service-mesh-for-microservices/>

Backup Slides

SAN JOSÉ STATE UNIVERSITY *powering* SILICON VALLEY